Highlights

$\tau\lambda$-Index: A Framework for Locating Rare Patterns in Repetitive Corpora[1]

Che-Wei Tsao, Jin-Jie Deng, Long-Qi Chen, Wing-Kai Hon, Dominik Köppl, Kunihiko Sadakane

- We extend the previous $\tau\lambda$-index into a framework for efficiently locating rare patterns in repetitive corpus.
- We propose a near-linear time algorithm to identify the rare information in the input corpus.
- Experimental results show that our index compares favorably with state-of-the-art indexing methods on highly

[1]Preliminary version of this work appeared in IEEE DCC Conference, 2024.

$\tau\lambda$-Index: A Framework for Locating Rare Patterns in Repetitive Corpora*

Che-Wei Tsao[a], Jin-Jie Deng, Long-Qi Chen[a], Wing-Kai Hon[a], Dominik Köppl[b], Kunihiko Sadakane[c]

[a]*Dept of Computer Science, National Tsing Hua University, Hsinchu, Taiwan (R.O.C.)*
[b]*Dept of Computer Science and Engineering, University of Yamanashi, Kofu, Japan*
[c]*Department of Mathematical Informatics, University of Tokyo, Tokyo, Japan*

Abstract

A substring P in a text T is called *rare* if it occurs at least τ_l and at most τ_u times, for two fixed integer parameters τ_l and τ_u. Rare patterns are useful in many applications, such as in the analysis of rare diseases from genome sequences. Unfortunately, solving rare pattern search with known full-text indexing techniques is not straightforward, especially for highly repetitive texts such as genome collections. Although several full-text indexes have already been developed to support time- and space-efficient full-text retrieval in compressed highly repetitive texts, we face two challenges for rare pattern search. First, a large fraction of the input data is likely to be redundant or not relevant for rare pattern search and thus the index size can be drastically reduced by extracting only information necessary for rare pattern search. Second, indexes with no efficient support for count queries, such as some Lempel–Ziv-based or grammar-based indexes, cannot easily determine whether a query pattern P is rare. To address these two challenges, we propose an index framework to locate rare patterns P of length at most λ. We also present a near-linear time method to identify the parts in T necessary for rare pattern search. Experimental results demonstrate that the $\tau\lambda$-index achieves competitive space-time trade-offs on highly repetitive corpora. In particular, for highly repetitive corpora such as real-world human DNA sequences, the $\tau\lambda$-index requires less space than the three self-indexes compared in our experiments, namely LZ77 [1], LPG [2], and the r-index [3]. Moreover, for indexes that do not support efficient count queries, our method offers significantly faster performance—often by orders of magnitude—with only a modest space overhead observed in certain corpora.

Keywords: Compressed Indexing, Pattern Matching, Rare Patterns

*Corresponding author at: Dept of Computer Science, National Tsing Hua University, No. 101, Section 2, Kuang Fu Road, Hsinchu, 30013, Taiwan (R.O.C.). Tel.: +886-3-5731307; E-mail: wkhon@cs.nthu.edu.tw

*Preliminary version of this work appeared in IEEE DCC Conference, 2024.

1. Introduction

Over the past three decades, genome sequencing technologies have advanced rapidly, beginning with the Human Genome Project [4], officially launched in 1990 with the goal of fully sequencing a single human genome. Since then, large-scale initiatives such as the 1000 Genomes Project [5] and the 100000 Genomes Project [6] have aimed to construct comprehensive datasets of human genetic variation and to investigate inter-individual differences in order to better understand their associations with genetic diseases. The rapid increase in genomic data presents significant challenges for storage, indexing, and analysis [7].

In parallel, significant progress has also been made in text indexing techniques that support efficient pattern matching. Let T be a text of length n, with characters drawn from an integer alphabet of size $\sigma = n^{O(1)}$. As early as the 1970s, the suffix tree [8, 9] was proposed, which supports near-linear time pattern matching using $O(n)$ words (or $O(n \log n)$ bits) of space. In 2000, the FM-index [10] was introduced, which utilized the Burrows-Wheeler Transform (BWT) [11] to reduce the space usage to at most $5nH_k(T) + o(n)$ bits, and later to $nH_k(T) + o(n)$ bits in 2007 [12], where $H_k(T)$ is the kth order empirical entropy of T. This made it feasible to index an entire human genome on RAM of a commodity machine at the same time: A pointer-based representation of the suffix tree built on a single human genome requires roughly 64GB of memory, while the FM-index fits in approximately 1GB.

However, indexing a single genome is no longer sufficient. For instance, analyzing rare diseases requires the ability to process a large number of human genomes simultaneously [13]. Given that human DNA sequences are approximately 99.9% identical [14], and that empirical entropy is largely insensitive to repetitiveness [15], the FM-index may not be the most space-efficient choice for highly repetitive data such as concatenated genome sequences. In such cases, alternative self-indexing methods based on other compression schemes may offer better space efficiency. These include Lempel–Ziv-based indexes [16, 17], grammar-compression-based indexes [18, 19], and run-length-compressed BWT indexes such as the r-index [3, 20].[2]

These self-indexes compress and store the text T based on different compression algorithms, and are designed to support full information retrieval from T. However, when the focus is on identifying rare information, such as in rare disease analysis, much of the stored data becomes redundant or not relevant to the rare information of interest. Here, we think about *rare information* as substrings with bounded lengths but low frequencies. With this definition in mind, a new idea is to reduce storage even further by discarding highly repetitive content and retaining only the rare information of interest. Another issue is that verifying whether a query pattern is rare can be time-consuming, especially for indexes that do not support efficient count queries. To this end, an index that supports the search and location of rare patterns, that is, patterns whose number of occurrences is below a given

[2]Very recently, an even smaller index called δ-SA was proposed [21]. It supports suffix array queries using optimal space (and is never asymptotically larger than the r-index), and can be constructed directly from the LZ77 parse in compressed time.

threshold τ_u, over a collection of similar genomes would be particularly valuable, especially in biomedical scenarios such as drug discovery for rare diseases [22]. Furthermore, the index should focus on functionally relevant regions of the genome, such as motifs (typically 5-20 bp [23]) and exons (typically 50-200 bp [14]). Hence, we set a parameter λ to limit the length of query patterns. This length is also well-matched to the local alignment phase of tools like BLAST [24], whose first step involves locating exact short matches, which are then extended to full alignments using seed-based strategies. Finally, to mitigate the risk of reporting patterns that are rare merely by chance, it is desirable to introduce a lower threshold τ_l on the number of occurrences to ensure statistical significance.

The $\tau\lambda$-index [25] was recently proposed to address these needs. However, its initial implementation relies on relatively naive, pointer-based data structures and requires that the full text T resides in memory at query time. In this work, we address these limitations and generalize the core idea of the $\tau\lambda$-index, which is the identification of rare information in a given text T, to other types of self-indexes. Thus, the $\tau\lambda$-index has become an adapter that incorporates a self-index. The main idea is to modify T so as to retain only the rare information, and then apply another index to the modified text.

In Section 3, we formally introduce the core idea, present a near-linear time algorithm for identifying the rare information of T, review the previous solution, and describe an improved alternative. In Section 4, we adapt three representative self-indexes, namely the r-index [3], a Lempel-Ziv-based index (LZ77) [1], and a grammar-based index (LPG) [2], to the $\tau\lambda$-index and evaluate their space and query performance.

2. Preliminaries

We define the input text as $T = T[1]T[2]...T[n]$ of length n and the query pattern as P of length m, where all characters are drawn from an alphabet of size σ. The last character of T is always the terminal symbol \$, which is the smallest symbol in the alphabet and appears only at position $T[n]$. For any $1 \leq i \leq j \leq n$, we denote the substring of T from position i to j as $T[i, j] = T[i]T[i + 1] \dots T[j]$. If $i > j$, we define $T[i, j]$ to be the empty string. The concatenation of a string S and a character c is denoted as Sc or cS, and $|S|$ represents the length of S.

We define two types of queries on a pattern P:

- *Count*(T, P): Returns the number of occurrences of P in T.
- *Locate*(T, P): Returns the set of all locations where P occurs in T.

Since *Count*$(T, P) = |$*Locate*$(T, P)|$, we can answer a count query via locate. Nevertheless, there are implementations that can answer count queries faster than locate queries.

In what follows, we define a third query, which is based on the two queries above. This query has three integer parameters τ_l, τ_u, and λ, and is defined as follows.

Definition 1. *Given a text T of length n and three integers τ_l, τ_u and λ. For any pattern P, the returned set of the query* $Locate_{\tau_l\tau_u\lambda}(T, P)$ *is:*

- *Locate*(T, P) *whenever* $\tau_l \leq$ *Count*$(T, P) \leq \tau_u$ *and* $|P| \leq \lambda$*;*
- *The empty set, otherwise.*

The goal of this article is to define an index on T that can efficiently compute the set $Locate_{\tau_l\tau_u\lambda}(T, P)$, where the integer parameters are given at indexing time, but P is given at query time.

2.1. eXtended Burrows-Wheeler Transform (XBWT)

Given an Aho–Corasick (AC) automaton [26], which stores several strings (the *keys*), and a query pattern P of length m, with each character drawn from an alphabet of size σ, we can determine whether P contains any key in $O(m \log \sigma)$ time if the children of each node are stored in a binary search tree. The AC-Automaton can be implemented as a pointer-based trie using $O(k \log \sigma \log k)$ bits, where k is the number of nodes in the trie. An example is shown in Figure 1(a).

To reduce the space usage, we can replace the AC-Automaton with an XBWT [27]. Let LT be the rooted, ordered, and labeled trie of the AC-Automaton. The XBWT is a compact transformation of LT that occupies $2k + k\lceil \log \sigma \rceil$ bits of space [27, Theorem 1] and supports both upward and downward navigation in $O(\log \sigma)$ time when implemented with a binary wavelet tree [28]. An illustrative example is shown in the left panel of Figure 1. The $\mathbf{\Pi}$ column encodes the edge labels from the internal nodes to the root, sorted in lexicographic order. The $\mathbf{L}$ column lists the edge labels to the children of each internal node, also sorted lexicographically according to the ordering of the corresponding entries in the $\mathbf{\Pi}$ column. The **Last** column marks the rightmost child of each internal node with a 1' and all others with 0'. The relative order of identical first characters in $\mathbf{\Pi}$ is preserved in the first characters of the corresponding entries in $\mathbf{L}$, enabling backward search in a manner analogous to that used in the BWT matrix.

Since failure links induce a tree structure, the *failure link tree* on the internal nodes having the original root as its root, Manzini discovered that the preorder ranks of the nodes in the failure link tree are the ranks of the nodes in $\mathbf{\Pi}$ [29, Lemma 4]. He therefore can represent the failure link tree with a balanced parentheses representation C of size $2k' + o(k')$ bits [29, Theorem 5], where k' is the number of internal nodes in LT. Suppose that the current entry in $\mathbf{\Pi}$ corresponds to the ith internal node (v_i). The failure link of v_i points to v_j so that $\mathbf{\Pi}[j]$ is the longest proper prefix of $\mathbf{\Pi}[i]$. The failure link can be traversed by the following three operations:

1. Let $k = \mathsf{select}_(C, i)$, which returns the position of the ith open parenthesis in C, corresponding to the ith internal node v_i.
2. Let $\ell = \mathsf{enclose}(C, k)$, which returns the position of the nearest open parenthesis that encloses the parenthesis at position k.
3. Let $j = \mathsf{rank}_(C, \ell)$, which returns the number of open parentheses in C up to and including position ℓ. The failure link of v_i therefore points to v_j.

If we implement C with a min-max tree [30], we can perform the three operations `select`, `enclose`, and `rank` in constant time [30, Theorem 1.1].

With this enhancement, the XBWT can simulate the behavior of an AC automaton. To construct the XBWT, we terminate each key with a special terminal symbol `$`, which is assumed to be the smallest symbol in the alphabet and does not appear elsewhere in the keys. Given a pattern P of length m, we conceptually traverse the AC automaton while matching P. Whenever we reach an internal node that has `$` as one of its children, we can conclude that P contains at least one complete key. The time required to determine whether P contains any key is $O(m \log \sigma)$, since at most $2m$ upward or downward edge traversals are performed [26, Theorem 2].

Figure 1 illustrates an example in which the XBWT is built on the set of strings $\{\texttt{aa}, \texttt{aba}, \texttt{ba}\}$. Figure 1(a) shows the corresponding trie, where each node is numbered according to the order in $\mathbf{\Pi}$ and the failure links are indicated by dashed arrows. Considering only the failure links, they induce a tree structure, as illustrated in Figure 1(b). Figure 1(c) shows the compact representation of the trie. Finally, Figure 1(d) presents the balanced-parentheses representation C of the failure link tree shown in Figure 1(b).

Given a query pattern $P = \texttt{abba}$, we start from v_1 and perform two backward search steps on the prefix `ab`, reaching v_7, which has no child labeled with `b`. Following the failure link once by performing `select`, `enclose`, and `rank`, which returns $(k, \ell, j) = (11, 10, 6)$, brings us to v_6, which does not have a child labeled `b`. Therefore, we follow the failure link again, which returns $(k, \ell, j) = (10, 1, 1)$, and brings us to v_1, which does have a child labeled `b`. From there, two additional backward search steps on the substring `ba` lead to v_4, which has a child labeled `$`, indicating that `ba` occurs in P.

2.2. r-index

The r-index [3] is the first BWT-based self-index that supports efficient locate queries using $O(r)$ words of space, where r is the number of character runs in the BWT of T. Due to the clustering effect of the BWT [31], the number of runs r tends to be small when T is highly repetitive. The r-index can be regarded as an extension of the RLFM-index [32], equipped with additional auxiliary structures to support locate queries. With $O(r)$ words of space, the r-index performs $\mathit{Count}(T, P)$ in $O(m \log \log_w(\sigma + n/r))$ time and $\mathit{Locate}(T, P)$ in $O((1 + \log \log_w(n/r)) \cdot \mathit{occ})$ time [3, Theorem 3.1], where w is the RAM word size in bits and $\mathit{occ} = \mathit{Count}(T, P)$.

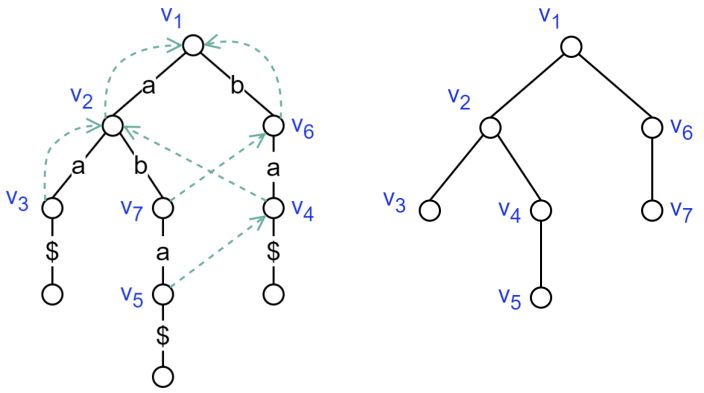
In this paper, the r-index implementation[3] we adopt requires $r(\log \sigma + (1+\epsilon) \log(n/r) + 2 \log n)$ bits of space. A count query takes $O(\frac{m}{\epsilon}(\log(n/r) + \log \sigma))$ time and (after counting) each occurrence can be located in $O(\log(n/r))$ time, where $\epsilon > 0$ is a fixed constant.

2.3. Lempel–Ziv 77 (LZ77)

The LZ77 factorization [33] is a left-to-right greedy parsing of the string T that splits T into phrases. Each phrase is represented by a tuple $\langle k, \ell, a \rangle$, where the phrase corresponds

[3] `https://github.com/nicolaprezza/r-index`

(a) (b) (c)



node	Last	L	Π
v_1	0	a	ε
	1	b	
v_2	0	a	a
	1	b	
v_3	1	\$	aa
v_4	1	\$	ab
v_5	1	\$	aba
v_6	1	a	b
v_7	1	a	ba

(d)

index	=	1	2	3	4	5	6	7	8	9	10	11	12	13	14
C	=	(	(	(	)	(	(	)	)	)	(	(	)	)	)
node	=	v_1	v_2	v_3		v_4	v_5				v_6	v_7			

Figure 1: The XBWT built on the input strings {aa, aba, ba}. Each input string is terminated by $ during construction. (a) The trie built from the strings, where failure links of internal nodes are indicated by dashed arrows. (b) The failure link tree derived from (a). (c) The compact representation of (a), consisting of the arrays **Last** and **L**. (d) The compact parenthesis representation C of (b).

to a substring $T[k, k+\ell-1]$ that occurs in a previous position in T, followed by a character a. By maximizing the length l at each step, LZ77 achieves a high compression ratio. Based on this compression scheme, several LZ77-based self-indexes have been developed [15, 16, 34].

In this paper, we adopt the implementation proposed by Claude et al. [1]. Although Claude et al. did not explicitly analyze its time and space complexities, they referred readers to the work of Kreft et al. [34] for a detailed discussion. The resulting index occupies $3z \log n + 5n \log \sigma + O(z) + o(n)$ bits of space and supports locate queries with a time complexity of $O(m^2 h + (m + occ) \log z)$ [34, Theorem 4.11], where z is the number of parsed phrases, h is the height of the parsing structure, and $occ = Count(T, P)$. A notable drawback of most LZ77-based self-indexes is that count queries are performed by executing full locate queries, rather than being supported directly.

2.4. LMS-based Parsing Grammar (LPG)

A grammar compression algorithm represents a text T as a tuple $\mathcal{G} = (V, \Sigma, \mathcal{R}, S)$, where Σ is the alphabet of terminal symbols, V is the set of nonterminal symbols, $\mathcal{R}$ is a set of production rules that transform nonterminals into strings over $(V \cup \Sigma)$, and $S \in V$ is the start symbol of $\mathcal{G}$.

The repetitiveness measure of a grammar compression algorithm is denoted by g, which is defined as the sum of the lengths of all right-hand sides of the rules in $\mathcal{R}$. Approximating

the minimal value of g within a small constant factor has been proven to be an NP-hard problem [35]. However, several heuristic implementations have been proposed [18, 15].

In this paper, we adapt the implementation proposed in [2]. The resulting index occupies $g \log n + (2 + \epsilon) g \log g$ bits of space and supports locate queries with a time complexity of $O((m \log m + occ) \log g)$ [2, Theorem 1], where $\epsilon > 0$ is a fixed constant and $occ = \mathit{Count}(T, P)$. Like Lempel–Ziv-based self-indexes, most grammar-compression-based self-indexes do not support count queries directly and instead answer them by performing locate queries.

3. $\tau\lambda$-Index

The $\tau\lambda$-index is designed to efficiently compute the set $\mathit{Locate}_{\tau_l \tau_u \lambda}(T, P)$. To this end, we are allowed to index T together with the three integer parameters τ_l, τ_u, and λ. At a high level, the index consists of two main components:

Component 1: an AC-automaton for storing all *minimal factors* of T (the precise definition is deferred to Section 3.1), and

Component 2: a data structure for storing the locations of rare information in T.

The AC-automaton will be used to filter out non-rare query patterns, leaving only the rare ones to be resolved by the second component. For the second component, we mask regions in the text that do not contain rare information with the hope that the modified text can be better compressed by the self-index than the original text T. A crucial idea is to identify rare information, for which we define a new regularity of substrings, called *minimal factors*. We structure this section as follows.

In Section 3.1, we introduce the concept of minimal factors, which is the core of rare information and are key to answering $\mathit{Locate}_{\tau_l \tau_u \lambda}(T, P)$. In Section 3.2, we present two methods for identifying all minimal factors of T. Sections 3.3 and 3.4 describe the $\tau\lambda$-index of the conference version and an improved variant, respectively. Both indexes consist of Component 1 and Component 2, but differ in their implementations. The overall idea of the $\tau\lambda$-index is to use Component 1 to filter out non-rare query patterns, leaving only rare patterns to be resolved by Component 2.

3.1. Minimal Factors

To answer $\mathit{Locate}_{\tau_l \tau_u \lambda}(T, P)$, we initially concentrate on the property of the number of occurrences of P in T. The following is a simple and valuable observation.

Lemma 1. *For any pattern P and a character c, $\mathit{Count}(T, cP) \leq \mathit{Count}(T, P)$ and $\mathit{Count}(T, Pc) \leq \mathit{Count}(T, P)$.*

Proof. Since P is a substring of cP and Pc, every occurrence of cP and Pc must contain an occurrence of P. Thus, $\mathit{Count}(T, cP) \leq \mathit{Count}(T, P)$ and $\mathit{Count}(T, Pc) \leq \mathit{Count}(T, P)$. □

Lemma 1 leads us to develop our main idea, *minimal factors*. The following is the definition:

Definition 2. *Given a text T and three integers τ_l, τ_u and λ, $MF_{\tau_l\tau_u\lambda}(T)$ denotes the set containing all substrings F of T that satisfy the following conditions:*

1. $|F| \leq \lambda$;
2. $\tau_l \leq \mathit{Count}(T, F) \leq \tau_u$;
3. $\mathit{Count}(T, S) > \tau_u$, *for every proper substring S of F;*

We refer to such an F as a minimal factor *of T.*

See Figure 2 for an example of $MF_{\tau_l\tau_u\lambda}(T)$. Minimal factors can be viewed as a generalization of *minimum unique substrings* [36], which correspond to $MF_{\tau_l\tau_u\lambda}(T)$ with $\tau_l = \tau_u = 1$ and $\lambda = n$. Note that any substring containing the terminal symbol `$` will not be identified as a minimal factor, because `$` serves as the terminator of T and does not relate to any other part of the text.

index	=	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
T	=	a	n	a	n	c	n	a	n	a	n	a	b	b	a	n	a	b	a	b	a	$
$\hat{T}$	=	#	#	a	n	c	n	a	#	#	#	a	b	b	a	n	#	b	a	b	#	$

Figure 2: Minimal factors and the corresponding $\hat{T}$ (defined in Section 3.4) for T = `anancnananabbanababa$` with $\tau_l = 1$, $\tau_u = 1$, and $\lambda = 3$. The minimal factors of T (`c`, `bb`, `ban`, and `bab`) are highlighted in red.

The following lemma gives us a linear upper bound for the total number of minimal factors.

Lemma 2. *Given a text T of length n, the set $MF_{\tau_l\tau_u\lambda}(T)$ contains at most n elements, and no two of them share the same start position or the same end position.*

Proof. If two minimal factors were to share the same start or end position, then they would either be identical or one would be a proper substring of the other, which contradicts the third condition of Definition 2. Therefore, each position of T can serve as the start position of at most one minimal factor. Consequently, $MF_{\tau_l\tau_u\lambda}(T)$ contains at most n elements. □

$MF_{\tau_l\tau_u\lambda}(T)$ is beneficial to answering $\mathit{Locate}_{\tau_l\tau_u\lambda}(T, P)$ in two ways. First, it allows us to quickly check whether $\mathit{Count}(T, P) \leq \tau_u$, helping us decide whether we need to perform the time-consuming locate queries. The corresponding theorem is presented below.

Theorem 3. *Let P be a query pattern. If there exists a substring S of P such that $S \in MF_{\tau_l\tau_u\lambda}(T)$, then $Count(T, P) \leq \tau_u$.*

Proof. By Definition 2, if $S \in MF_{\tau_l\tau_u\lambda}(T)$, then $Count(T, S) \leq \tau_u$. Since S is a substring of P, it follows from Lemma 1 that $Count(T, P) \leq Count(T, S) \leq \tau_u$. □

Second, $MF_{\tau_l\tau_u\lambda}(T)$ also indicates which part of T should be saved for future queries. The definition of these parts is given below.

Definition 3. *Let T be a text. For any minimal factor $T[i, j] \in MF_{\tau_l\tau_u\lambda}(T)$, we call the substring $T[j - \lambda + 1, i + \lambda - 1]$ to be* related *to this minimal factor.*

By definition, indexing all *related* substrings is sufficient to answer the $\tau\lambda$-locate query. Our idea is to implement two data structures to construct the $\tau\lambda$-index. The first is used to store $MF_{\tau_l\tau_u\lambda}(T)$ and verify whether P is rare. The second is used to store the substrings of T which are *related* to at least one minimal factor and answers $Locate_{\tau_l\tau_u\lambda}(T, P)$.

3.2. Identification of Minimal Factors

We can compute $MF_{\tau_l\tau_u\lambda}(T)$ using two FM-indexes [12]: one built on T and the other on T^{rev}, the reverse of T. We denote the two FM-indexes as $FM(T)$ and $FM(T^{\text{rev}})$. The algorithm is described in Algorithm 1. To facilitate the proof, we introduce the following definition:

Definition 4. *A minimal-factor candidate F' is a substring of T that satisfies all conditions of Definition 2 except the requirement $Count(T, F') \geq \tau_l$. In other words, a minimal-factor candidate F' satisfies: (1) $|F'| \leq \lambda$; (2) $Count(T, F') \leq \tau_u$; and (3) $Count(T, S) > \tau_u$, for every proper substring S of F'.*

The high-level concept of Algorithm 1 is to locate the leftmost minimal-factor candidate within each substring $T[i,\ i + \lambda - 1]$ for $1 \leq i \leq n$. We introduce another integer $j = i$ and extend j one position at a time until either

Condition 1: $Count(T, T[i, j]) \leq \tau_u$, or

Condition 2: $j > i + \lambda - 1$ holds.

If Condition 1 holds, we conclude that j is the end position of the leftmost minimal-factor candidate. We then introduce another integer $i' = j$ and extend i' leftward one position at a time until $Count(T, T[i', j]) \leq \tau_u$. By Definition 4, i' is the start position of the leftmost minimal-factor candidate. Finally, we verify whether $Count(T, T[i', j]) \geq \tau_l$ to confirm that $T[i', j]$ is an element of $MF_{\tau_l\tau_u\lambda}(T)$. If Condition 2 holds, then $T[i,\ i + \lambda - 1]$ contains no minimal-factor candidate, and we increment i by one to process the next substring.

Theorem 4. *Algorithm 1 computes $MF_{\tau_l\tau_u\lambda}(T)$ in $O(n\lambda \log \sigma)$ time.*

Algorithm 1: Identify $MF_{\tau_l\tau_u\lambda}(T)$ using $FM(T)$ and $FM(T^{rev})$

```
1  i ← 1;
2  while i ≤ n do
3      j ← i;
4      while j ≤ i + λ − 1 and Count(T, T[i, j]) > τ_u do
5          j ← j + 1;
6      if Count(T, T[i, j]) ≤ τ_u then
7          i' ← j;
8          while Count(T, T[i', j]) > τ_u do
9              i' ← i' − 1;
10         if Count(T, T[i', j]) ≥ τ_l then
11             insert T[i', j] into MF_{τ_l τ_u λ}(T);
12         i ← i' + 1;
13     else
14         i ← i + 1;
```

Proof. The correctness relies on the invariant that

every minimal factor starting before i has already been identified. (I1)

We begin with $i = 1$. In this initial configuration, no minimal factor starts before $i = 1$, so Invariant (I1) trivially holds. We prove Invariant (I1) for $i \geq 2$ by detecting the leftmost minimal-factor candidate in $T[i, n]$, which we denote by $F' = T[s, t]$. Our goal is to compute s and t and, then check whether F' is actually a minimal factor. For that, we consider the following two cases.

- Case (a) $t \leq i + \lambda - 1$.

  Since $T[s, t]$ is a substring of $T[i,\, i+\lambda-1]$, we have $\mathit{Count}(T, T[i,\, i+\lambda-1]) \leq \tau_u$ by Lemma 1. The inner while loop (line 4) therefore stops when $\mathit{Count}(T, T[i, j]) \leq \tau_u$. Moreover, since $\mathit{Count}(T, T[i, j']) > \tau_u$ for all $i \leq j' < j$ and $\mathit{Count}(T, T[i, j]) \leq \tau_u$, it follows that a suffix of $T[i, j]$ is the minimal-factor candidate F', and hence $j = t$. This triggers line 6, after which we extend i' leftward until $\mathit{Count}(T, T[i', j]) \leq \tau_u$ (line 8). By Definition 4, we conclude that $T[i', j] = T[s, t]$. Hence, we have detected F'. It remains to verify whether F' is a minimal factor. For that it suffices to check whether $\mathit{Count}(T, T[i', j]) \geq \tau_l$ holds. If $F' \notin MF_{\tau_l\tau_u\lambda}(T)$, by Lemma 1, any substring beginning with $T[s, t]$ occurs fewer than τ_l times in T and therefore cannot be an element of $MF_{\tau_l\tau_u\lambda}(T)$. In any case, we set $i = i' + 1$ after this verification (line 12). Thus, Invariant (I1) holds, since there is no minimal factor starting in the interval $[i, i']$; otherwise, this would contradict the choice of F'.

- Case (b) $i + \lambda - 1 < t$.

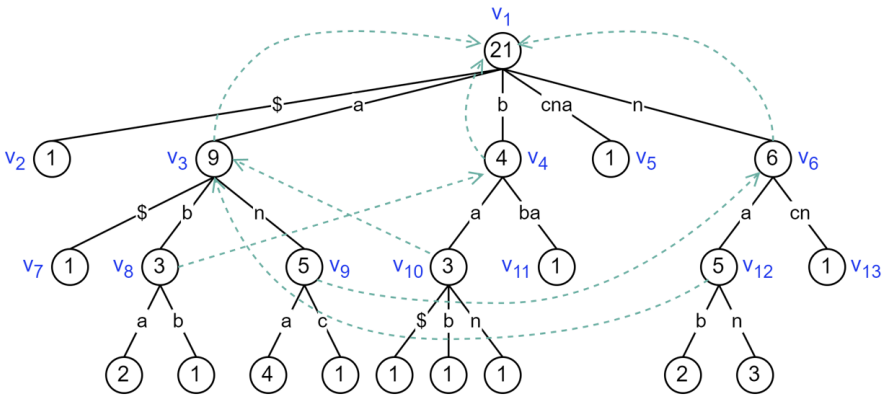


Figure 3: The λ-TST of $T = \texttt{anancnananabbanababa\$}$, with $\lambda = 3$. Gray dashed lines represent suffix links. The number shown at each node v denotes v.count, and the blue notation indicates the node index.

Since $T[s,t]$ is not a substring of $T[i, i+\lambda-1]$, we conclude that $\mathit{Count}(T, T[i, i+\lambda-1]) > \tau_u$, and the inner while loop (line 4) stops when $j > i+\lambda-1$. This triggers line 13, and the algorithm increments i by one without missing any minimal factor. Thus, Invariant (I1) holds.

There are at most n iterations of the outer while loop (line 2), and each iteration performs at most 2λ iterations of the inner while loops (lines 4 and 8). The two inner while loops can be implemented using backward search on $\mathit{FM}(T^{\mathrm{rev}})$ and $\mathit{FM}(T)$, respectively, where the value $\mathit{Count}(T, T[i,j])$ and $\mathit{Count}(T, T[i',j])$ in each iteration correspond to the size of the associated *SA-interval* [12, Figure 3]. If $\mathit{FM}(T^{\mathrm{rev}})$ and $\mathit{FM}(T)$ are implemented using a binary wavelet tree [28], each update of the SA-interval during backward search can be obtained in $O(\log\sigma)$ time [12, Theorem 4.4]. Therefore, the overall time complexity is $O(n\lambda\log\sigma)$. □

The approach of Theorem 4 becomes unattractive for large values of λ. In what follows, we propose an alternative $O(n\log\sigma)$-time approach to identify $\mathit{MF}_{\tau_l\tau_u\lambda}(T)$ using the λ-truncated-factor tree (λ-TST) [37], which is a pruned suffix tree of limited height λ. An example is shown in Figure 3. The main idea is to modify the textbook algorithm of Gusfield [38, APL 8] for computing matching statistics on suffix trees. Broadly speaking, *matching statistics* asks for the longest matching prefix of $P[i,m]$ in T for each position i, for a given pattern P of length m. Gusfield's algorithm uses two operations: `extend` and `contract`. The `extend` operation traverses downward along the suffix tree according to the next character of P, while the `contract` operation traverses upward via suffix links. By maintaining two positions i and j of P to represent the matched substring $P[i,j]$ together with the current node v_{cur} in the suffix tree, the algorithm computes the matching statistics of P in $O(m\log\sigma)$ time if the children of each node are stored in a binary search tree. We

modify this algorithm to identify $MF_{\tau_l\tau_u\lambda}(T)$ from λ-TST by matching T with itself. For explaining our modification, we introduce the following terminology:

- v: a node in λ-TST.
- v.child(c): the node pointed to by the edge from v whose label begins with the character c.
- v.length(c): the length of the edge from v whose label begins with the character c.
- `string-label`(v): the string obtained by concatenating the edge labels along the path from the root to v.
- $SL(v)$: the suffix link of v, which points to the node u such that `string-label`(u) equals `string-label`(v) with its first character removed. The suffix link of the root points to itself. We do not consider suffix links for leaf nodes, since Algorithm 2 never traverses to a leaf.
- v.count: the value $\mathit{Count}(T, \texttt{string-label}(v))$. During the construction of the λ-TST, the value v_ℓ.count of each leaf v_ℓ is incremented every time its corresponding suffix is visited. After the construction, the values v_{i}.count for all internal nodes v_i are computed by a bottom-up traversal of the tree, where each node aggregates the occurrence counts of its children.

Algorithm 2 describes how to identify $MF_{\tau_l\tau_u\lambda}(T)$ from λ-TST. The high-level idea is as follows. We introduce two integers i and j to represent the substring $T[i, j]$, and maintain a node v_{cur} to represent the current node of λ-TST, with the invariants that

Invariant 1: $T[i, j-1] = \texttt{string-label}(v_{\mathrm{cur}})$ and

Invariant 2: $\mathit{Count}(T, T[i, j]) = v_{\mathrm{cur}}.\mathrm{child}(T[j]).\mathrm{count}$.

By Invariant 1, $T[j]$ is always the first character of an edge. We first fix i and increment j until either

Condition 1: $\mathit{Count}(T, T[i, j]) \leq \tau_u$, or

Condition 2: $j > i + \lambda - 1$

holds. While increasing j, we traverse downward along λ-TST to maintain the invariants.

If Condition 1 holds, it means that $T[i, j]$ contains a minimal-factor candidate $F' = T[s, t]$ and that $j = t$. In this case, we fix j and increment i while traversing upward in λ-TST via suffix links until s is identified. Finally, we verify whether $\mathit{Count}(T, F') \geq \tau_l$ to confirm that F' is an element of $MF_{\tau_l\tau_u\lambda}(T)$. If Condition 2 holds, it means that $T[i,\, i+\lambda-1]$ contains no minimal-factor candidate F', and we then traverse upward along λ-TST via the suffix link and increment i by one for the next iteration.

Theorem 5. *Algorithm 2 identifies $MF_{\tau_l\tau_u\lambda}(T)$ in $O(n \log \sigma)$ time.*

Algorithm 2: Identify $MF_{\tau_l\tau_u\lambda}(T)$ using λ-deep factor tree

```
1  i ← 1, j ← 1, v_cur ← root;
2  while i ≤ n and j ≤ n do
3      if v_cur.child(T[j]).count ≤ τ_u then
4          while v_cur.child(T[j]).count ≤ τ_u and v_cur is not the root do
5              i ← i + 1;
6              v_prev ← v_cur;
7              v_cur ← SL(v_cur);
8          if v_cur.child(T[j]).count > τ_u then
9              if v_prev.child(T[j]).count ≥ τ_l then
10                 Insert T[i − 1, j] into MF_{τ_l τ_u λ}(T);
11         else
12             if v_cur.child(T[j]).count ≥ τ_l then
13                 Insert T[i, j] into MF_{τ_l τ_u λ}(T);
14             i ← i + 1; j ← j + 1;
15     else if v_cur.child(T[j]) is not a leaf then
16         j ← j + v_cur.length(T[j]);
17         v_cur ← v_cur.child(T[j]);
18     else
19         i ← i + 1; j ← max(i, j);
20         v_cur ← SL(v_cur);
```

Proof. Analogous to the proof of Theorem 4, the correctness relies again on Invariant (I1). We begin with $i = j = 1$ and set v_{cur} to be the root of λ-TST. In this initial configuration, no minimal factor starts before $i = 1$, so Invariant (I1) holds. For $i \geq 2$, we prove Invariant (I1) by computing the leftmost minimal-factor candidate in $T[i, n]$, which we denote by $F' = T[s, t]$. Having computed s and t, it remains to verify whether F' is actually a minimal factor. To this end, we monitor whether $v_{\text{cur}}.\text{child}(T[j]).\text{count} \leq \tau_u$ and whether $v_{\text{cur}}.\text{child}(T[j])$ is a leaf. This leads us to the following three cases, which are illustrated in Figure 4.

- Case (a) $j = t$.

  Since $T[s, t]$ is a substring of $T[i, j]$, we have $v_{\text{cur}}.\text{child}(T[j]).\text{count} = \mathit{Count}(T, T[i, j]) \leq \tau_u$, which triggers line 3. We then fix j and increment i (line 5) while traversing upward along λ-TST via suffix links until either

  Condition 1: $v_{\text{cur}}.\text{child}(T[j]).\text{count} > \tau_u$ or

  Condition 2: v_{cur} is the root.

  At each step of this traversal, we record the previously visited node in v_{prev} (line 6).

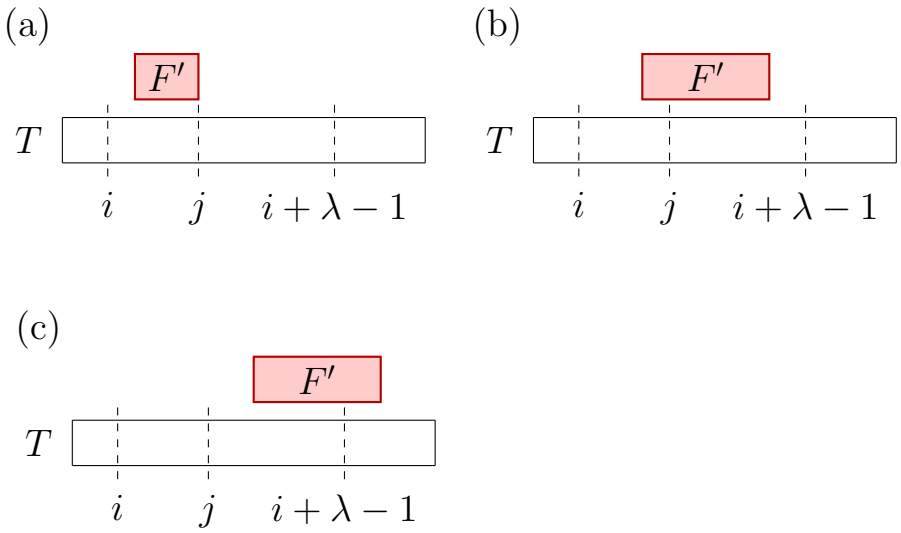


Figure 4: Illustration of the positions i, j, and $i+\lambda-1$ with the leftmost minimal-factor candidate F'. Cases (a), (b), and (c) correspond to the proof of Theorem 5.

If Condition 1 holds (line 8), we obtain $T[i-1,j] = T[s,t]$ by Definition 4. We then check the condition

$$\mathit{Count}(T, T[s,t]) = v_{\text{prev}}.\text{child}(T[j]).\text{count} \geq \tau_l$$

to determine whether $T[s,t]$ is an element of $MF_{\tau_l\tau_u\lambda}(T)$ (line 9).

If Condition 2 holds (line 11), we have traversed to the root without finding any node satisfying Condition 1. We have $i = j$ and $T[i,j] = T[s,t] = T[s]$ is a single character. In this case we verify whether $\mathit{Count}(T, T[s]) = v_{\text{cur}}.\text{child}(T[j]).\text{count} \geq \tau_l$ to determine whether $T[s]$ belongs to $MF_{\tau_l\tau_u\lambda}(T)$ (line 12). Finally, both i and j are incremented by one (line 14).

For any of both conditions, we set $i = s+1$ after the processing. If $F' = T[s,t] \notin MF_{\tau_l\tau_u\lambda}(T)$, then, by Lemma 1, any substring beginning with $T[s,t]$ occurs fewer than τ_l in T times and therefore cannot be an element of $MF_{\tau_l\tau_u\lambda}(T)$. Thus, Invariant (I1) holds, since there is no minimal factor starting in the interval $[i,i']$; otherwise, this would contradict the choice of F'.

- Case (b) $j < t \leq i+\lambda-1$.

  Since $F' = T[s,t]$ is the leftmost minimal-factor candidate in $T[i,n]$ and is not a substring of $T[i,j]$, we have $v_{\text{cur}}.\text{child}(T[j]).\text{count} > \tau_u$ and $\mathit{Count}(T, T[i,j]) > \mathit{Count}(T, T[i,t])$. Moreover, $v_{\text{cur}}.\text{child}(T[j])$ cannot be a leaf; otherwise, $T[j]$ and $T[t]$ would lie on the same edge, implying $\mathit{Count}(T, T[i,j]) = \mathit{Count}(T, T[i,t])$, a contradiction. Therefore, at each iteration of the outer while loop (line 2), the algorithm executes line 15 and continues extending j until $j = t$, at which point Case (a) will be triggered. We ensure that the incrementation of j stops exactly at t because $T[j]$

is always the first character of an edge. Since only j is incremented while i remains unchanged in Case (b), Invariant (I1) is preserved.

- Case (c) $i + \lambda - 1 < t$.

  Since $T[s, t]$ is the leftmost minimal-factor candidate and is not a substring of $T[i, j]$, we have $v_{\mathrm{cur}}.\mathrm{child}(T[j]).\mathrm{count} > \tau_u$. If $v_{\mathrm{cur}}.\mathrm{child}(T[j])$ is a leaf, line 18 is triggered; otherwise, line 15 is executed and j is extended.

  In the former case (line 18), we obtain $\mathit{Count}(T, T[i, i + \lambda - 1]) > \tau_u$, indicating that no minimal factor can start at i. Therefore, the algorithm increments i by one (line 19), and Invariant (I1) is preserved. Note that j is updated to $\max(i, j)$ (line 19) to handle the situation where $i + 1 > j$, which can only occur when v_{cur} is the root.

  In the latter case (line 15), only j is incremented while i remains unchanged, so Invariant (I1) continues to hold.

Time Complexity. Constructing the λ-TST takes $O(n \log \sigma)$ time if the children of each node are stored in a binary search tree. Subsequently, traversing the λ-TST takes $O(\log \sigma)$ time per step. Since each iteration of the while loops (lines 2 and 4) increments either i or j by at least one, Algorithm 2 runs in $O(n \log \sigma)$ time. □

As an example, we illustrate a part of the process using Figures 2 and 3. We use a tuple (i, j, v_{cur}) to represent the substring $T[i, j]$ and the current node v_{cur} in λ-TST. Assume the current state is $(10, 12, v_{12})$; it is then updated to $(11, 12, v_3)$ (line 18), then to $(11, 13, v_8)$ (line 15), then to $(12, 13, v_4)$ (line 4), and finally to $(13, 13, v_1)$ (line 4), at which point we identify $T[12, 13]$ as a minimal factor (line 10).

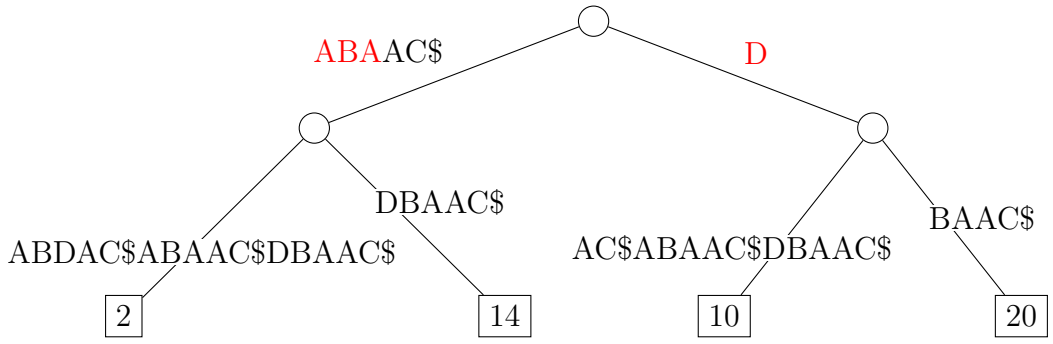
3.3. Previous Solution

In the conference version of this work [25], we proposed a relatively simple solution. To answer $\mathit{Locate}_{\tau_l \tau_u \lambda}(T, P)$, we used two kinds of data structures.

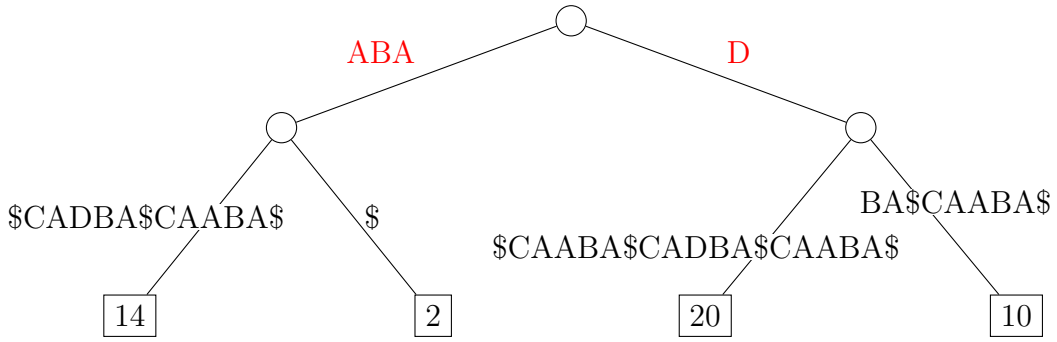
1. The first is a pointer-based AC-Automaton [26], which stores $MF_{\tau_l \tau_u \lambda}(T)$, to identify whether P is rare by the Aho–Corasick algorithm. The time complexity for identifying a minimal factor in P is $O(m)$ and the space complexity is $O(\min(n, |MF_{\tau_l \tau_u \lambda}(T)|) \lambda \log \sigma \log n)$ bits.

2. The second consists of two location trees (forward and reverse), which are essentially two pointer-based pruned suffix trees for answering the $\mathit{Locate}(T, P)$. The two location trees are built as follows: For each substring $T[t_i, t_j] \in MF_{\tau_l \tau_u \lambda}(T)$, we insert $T[t_i, n]$ into the forward location tree and store the reverse of $T[1, t_j]$ into the reverse location tree. After insertion, we store t_i in their corresponding leaf node. An example is shown in Figure 5.

With these two location trees, when performing pattern matching, after finding a minimal factor $F = P[p_i, p_j]$ in the pattern P, we search for the substring $P[p_i, m]$ in the



(a) Location Tree



(b) Reverse Location Tree

Figure 5: The location tree and reverse location tree constructed from $T =$ `$ABAAC$ABDAC$ABAAC$DBAAC$` with $\tau_l = 1$, $\tau_u = 2$, and $\lambda = 3$. The minimal factors of T are `ABA` and `D`, which are colored in red in the figure. The leaves point to the start position corresponding of each minimal factor. Note that any substring containing the terminal symbol $ will not be identified as a minimal factor.

forward location tree. We traverse the location tree from its root along the path label for this substring. If the substring exists, we arrive at the locus node v in the location tree that represents this substring. At this point, we simply perform a depth-first search in the subtree rooted at node v and return all the locations loc_f recorded on the leaf nodes within that subtree. Similarly, we also match the reverse string of $P[1, p_j]$ in the reverse location tree and get all the locations loc_r. Let loc_k be the intersection of loc_f and loc_r. For each position k in loc_k, if $P = T[k - p_i + 1, k - p_i + m]$, then we add $k - p_i + 1$ in the candidate results, loc_c. Finally, check whether the size of loc_c is less than τ_l. If so, return the empty set; otherwise, return loc_c.

Matching $P[p_i, m]$ and the reverse of $P[1, p_j]$ in the two location trees takes $O(m)$ time, finding locations in the subtree rooted at the locus node takes $O(Count(T, F)) = O(\tau_u)$ time, so the total time spent on location matching is $O(m+\tau_u)$ time. Regarding the space, the two pruned-suffix trees take $O(\tau_u |MF_{\tau_l\tau_u\lambda}(T)| \log n)$ bits, which is of $O(n \log n)$ bits.

3.4. Improved Solution

The previous solution has two significant drawbacks, both of which contribute to increased space usage. First, the AC automaton and the two location trees are pointer-based data structures, which can be replaced with succinct alternatives. Second, queries on the location trees require the full text T to reside in RAM at query time. In our improved design, both data structures are replaced to address these issues.

We define some terminologies for clarity in the subsequent discussion.

Definition 5. *Given a text T and $MF_{\tau_l\tau_u\lambda}(T)$, we define the following notation. First, let # be a symbol that neither appears in the text T nor in any pattern we will query. Then define $\hat{T}$ as the text obtained by replacing each character in the substrings of T that are not* related *(Definition 3) to any element of $MF_{\tau_l\tau_u\lambda}(T)$ with #. An example is shown in Figure 2. Finally, let $I(T)$ and $I(\hat{T})$ denote the self-index on T and $\hat{T}$, respectively.*

The AC-Automaton is represented with an XBWT, which also stores $MF_{\tau_l\tau_u\lambda}(T)$ while reducing the space usage from $O(\min(n, |MF_{\tau_l\tau_u\lambda}(T)|)\lambda \log \sigma \log n)$ bits to $O(\min(n, |MF_{\tau_l\tau_u\lambda}(T)|)\lambda \log \sigma)$ bits. The time complexity increases from $O(m)$ to $O(m \log \sigma)$.

The two location trees are replaced with $I(\hat{T})$. While the $\tau\lambda$-index allows $I(\hat{T})$ to be any self-index that supports locate queries, our current implementation supports the r-index [3], LZ77 [1], and LPG [2] as instantiations of $I(\hat{T})$. The space and time complexities depend on the choice of $I(\hat{T})$, as discussed in the preliminaries.

Theoretically, replacing T with $\hat{T}$ is not guaranteed to decrease any repetitiveness measure, thereby making the index size smaller. Our hypothesis is that when the masking ratio is sufficiently high, the repetitiveness measure of $\hat{T}$ is likely to be smaller than that of T. In Section 4, we apply this method to different types of corpora and examine its varying effects.

Algorithm 3 describes how the improved $\tau\lambda$-index answers $Locate_{\tau_l\tau_u\lambda}(T, P)$. In Step 1, we first check whether $|P| \leq \lambda$. If so, we proceed to Step 2; otherwise, the empty set is returned. Next, the XBWT is used to verify whether P contains any element of $MF_{\tau_l\tau_u\lambda}(T)$.

If it does, we conclude that $\mathit{Count}(T, P) \leq \tau_u$ by Lemma 1 and proceed with the query. Otherwise, since $\mathit{Count}(T, P) < \tau_l$ or $\mathit{Count}(T, P) > \tau_u$, the empty set is returned. In Step 3, we perform a count query using $I(\hat{T})$ to check whether $\mathit{Count}(\hat{T}, P) \geq \tau_l$. If this condition holds, we further use $I(\hat{T})$ to answer $\mathit{Locate}(\hat{T}, P)$, which in this case equals $\mathit{Locate}_{\tau_l\tau_u\lambda}(T, P)$. Otherwise, the empty set is returned.

It is worth mentioning that Step 2 cannot be modified to rely on the count query performed by $I(\hat{T})$ to verify whether $\mathit{Count}(T, P) \leq \tau_u$, since it may occur that $\mathit{Count}(\hat{T}, P) \leq \tau_u < \mathit{Count}(T, P)$ for some pattern P. In such cases, P might be incorrectly accepted as valid for $\mathit{Locate}_{\tau_l\tau_u\lambda}(T, P)$, leading to an incorrect result. Figure 2 illustrates this issue. Since $\mathit{Count}(T, \texttt{na}) = 5$, the correct answer to $\mathit{Locate}_{\tau_l\tau_u\lambda}(T, \texttt{na})$ should be the empty set. However, if Step 2 is evaluated using $\mathit{Count}(\hat{T}, \texttt{na}) = 1 \leq \tau_u$, the pattern `na` would incorrectly pass the test. Consequently, Step 3 would return the incorrect result $\mathit{Locate}_{\tau_l\tau_u\lambda}(T, \texttt{na}) = 6$.

Algorithm 3: Locate query in the $\tau\lambda$-index

1. If $|P| > \lambda$, return the empty set.
2. If P contains no substring that belongs to $\mathit{MF}_{\tau_l\tau_u\lambda}(T)$, return the empty set.
3. If $\mathit{Count}(\hat{T}, P) \geq \tau_l$, return $\mathit{Locate}(\hat{T}, P)$. Otherwise, return the empty set.

4. Experiments

In this section, we evaluate the practical performance of our proposed $\tau\lambda$-index. We compare it against three baseline indexes: the r-index, LZ77, and LPG, all built on T. The experiments are conducted on various corpora, including both artificial and real-world datasets. We analyze the space usage and query performance, focusing on the time taken for answering $\mathit{Locate}_{\tau_l\tau_u\lambda}(T, P)$ and the index size.

To facilitate the presentation of the experimental results, we first clarify the notation used to distinguish different self-index instantiations. When specifying the type of self-index used for $I(T)$ and $I(\hat{T})$, we replace I with the corresponding index type. For example, r-index(T) denotes the r-index built on T, while r-index($\hat{T}$) denotes the r-index built on $\hat{T}$. We use $\tau\lambda(I(\hat{T}))$ to denote the $\tau\lambda$-index constructed with a self-index of type I on $\hat{T}$. For instance, $\tau\lambda$(LZ77($\hat{T}$)) denotes the $\tau\lambda$-index consisting of the XBWT and LZ77($\hat{T}$).

4.1. Experiment Setup

We conducted our experiments on an Intel Xeon E5-1620 v4 machine clocked at 3.50GHz running Ubuntu 23.04 LTS. The program was compiled using `g++ 12.3.0` with options `-std=c++17 -O3` and with the sdsl-lite library [39]. Our implementation can be found at the following link: `https://github.com/CheWeiTsao/tau-lambda-index`.

To evaluate time and space usage, we adapted the r-index[4] (BWT-based index), LZ77[5] (Lempel-Ziv-based index) and LPG[6] (grammar-based index) as $I(\hat{T})$. These three indexes were also built directly on T and serve as baselines in our experiments. To adapt these indexes to answer $Locate_{\tau_l\tau_u\lambda}(T, P)$, in a manner similar to how the $\tau\lambda$-index operates, we applied slight modifications. The r-index performs a locate query only if the preceding count query confirms that $\tau_l \leq Count(T, P) \leq \tau_u$. For LZ77 and LPG, the locate query is terminated early and the empty set is returned if the number of found occurrences of P exceeds τ_u.

The corpora used in this study are listed in Table 1. Each corpus is composed of several similar samples, and the sample count is defined as the number of samples it contains. The first three are from Pizza&Chili[7]. The corpora *dblp* and *english* are constructed by repeatedly duplicating the same initial text 100 times, where each new copy (sample) is generated from the previous one with a random mutation applied. The mutation rate of *dblp* is set to 0.001, while that of *english* is set to 0.01. The corpus *dna_pseudoreal* is an artificial human DNA sequence [3], constructed by repeating a 1000-length DNA segment 629140 times, with a mutation rate of 0.001 applied independently to each character. The last corpus, *dna_real*, is constructed by concatenating DNA sequences from the same genomic position across 2000 individuals in the 1000 Genomes Project [5]. The selected region is chr1:144571600–144623999, which has a SNP density of approximately 2.58 SNPs per kb, which is comparable to the average SNP density of the human genome (around 1 SNP per kb) [40, 41]. Each sample in a corpus is delimited by a unique symbol (we use the ASCII value 127). We apply this delimiter to all corpora except *influenza*, for which no clear boundary between samples could be identified.

We set $\tau_l = 1$; $\tau_u \in \{1\%, 2\%, 3\%, 4\%\}$ of the sample count; and $\lambda \in \{20, 60, 100, 200\}$. For each corpus, we prepare two types of query pattern: (1) those containing at least one element of $MF_{\tau_l\tau_u\lambda}(T)$, and (2) those containing no element of $MF_{\tau_l\tau_u\lambda}(T)$. For each combination of τ_l, τ_u, and λ, we conducted experiments with 100 randomly generated query patterns. Each query pattern has length λ and is guaranteed to occur in T.

The index size was evaluated in terms of serialized space. The query time (wall-clock time) was measured using the C++ `<chrono>` library.

4.2. Masked Ratio and Repetitiveness Measures

The masked ratio and the corresponding repetitiveness measures for different corpora with various values of τ_u and λ are presented in Figure 6. The masked ratio of $\hat{T}$ is defined as $Count(\hat{T}, \#)/|T|$. To highlight the trend, each combination of τ_u and λ is colored according to its value, where deeper orange indicates a higher masked ratio.

For clarity, each repetitiveness measure in the right panel is normalized by the corresponding value of T. A value less than 1 indicates that the repetitiveness measure of $\hat{T}$

[4] `https://github.com/nicolaprezza/r-index`

[5] `https://github.com/migumar2/uiHRDC`

[6] `https://github.com/ddiazdom/LMS-based-self-index/tree/LPG_grid`

[7] `https://pizzachili.dcc.uchile.cl/repcorpus.html`

Table 1: Dataset Statistics

	dblp	*english*	*influenza*	*dna_pseudoreal*	*dna_real*
$\|\Sigma\|$	90	107	15	5	11
sample count	100	100	78,041	629,140	2,000
corpus size (MiB)	104	104	154	629	100
r	270,203	1,449,515	3,022,822	1,286,534	39,206
z	98,308	404,540	1,115,408	832,868	11,132
g	455,510	2,194,847	3,304,035	1,443,200	24,554
			(one sample)		
r-index size (MiB)	1.287	3.253	-	0.009	0.200
LZ77 size (MiB)	0.792	1.890	-	0.002	0.089
LPG size (MiB)	1.502	2.613	-	0.003	0.109
			(whole file)		
r-index size (MiB)	2.758	13.723	27.442	13.123	0.404
LZ77 size (MiB)	1.006	3.952	11.182	8.404	0.108
LPG size (MiB)	3.639	18.585	28.568	12.663	0.185

is lower than that of T, and vice versa. Note that a lower repetitiveness measure implies higher repetitiveness of the text. As described in the preliminaries, the space required by the three self-indexes used in our experiments is related to their corresponding repetitiveness measures (r for the r-index, z for LZ77, and g for LPG); hence, a lower repetitiveness measure indicates better compression efficiency.

All the masked ratios of *dblp* exceed 98%, and their repetitiveness measures are consistently lower than the baseline. In contrast, *english*, which has a mutation rate ten times higher than that of *dblp*, exhibits lower masked ratios. Although repetitiveness measures tend to be higher when the masked ratio is low, they drop sharply once the masked ratio exceeds 96%. All the masked ratios of *influenza* are below 40%, which we attribute to the naturally high mutation rate of this virus [42]. Its repetitiveness measures increase slightly as the masked ratio increases. For *dna_pseudoreal*, it exhibits a trend similar to that of *english*, characterized by overall lower masked ratios and a sharp increase in repetitiveness measures at low masked ratios. The difference between the two is that the repetitiveness measures increase much more significantly in *dna_pseudoreal*. This discrepancy may result from the difference in mutation methods: in *english*, each new copy is generated by mutating the previous one, leading to accumulated changes over time. In contrast, *dna_pseudoreal* applies random mutations independently to each character of the original sequence. For *dna_real*, it exhibits a similar trend to *dblp*, which is characterized by consistently high masked ratios and sharply decreasing repetitiveness measures.

In summary, while the repetitiveness measures do not exhibit a clearly predictable trend at low masked ratios, they consistently decrease once the masked ratio exceeds a

high threshold (e.g., 97%).

4.3. Space Usage

The space usage of indexes is presented in Figure 7. To improve clarity, the value is expressed as percentage difference (%) relative to $I(T)$. In other words, space($\tau\lambda(I(\hat{T}))$)/space($I(T)$)$-$1, where space($\tau\lambda(I(\hat{T}))$) represents the serialized space of $\tau\lambda(I(\hat{T}))$. As shown in Figure 8, the usage of XBWT space is expressed as percentage (%) relative to $\tau\lambda(I(\hat{T}))$, i.e., space(XBWT)/space($\tau\lambda(I(\hat{T}))$). To highlight the trend, each combination of τ_u and λ is colored according to its value: deeper orange indicates higher space usage than the baseline, whereas deeper blue indicates lower space usage. Therefore, deeper blue corresponds to better performance in Figure 7.

The index space is determined by the two data structures of the $\tau\lambda$-index, namely, the XBWT and $I(\hat{T})$. Since the size of the XBWT is positively correlated with λ, an obvious trend can be observed in *dblp*, *english*, and *dna_real*. This trend is less apparent in *influenza* and *dna_pseudoreal* due to the relatively small contribution of the XBWT to the overall space, as shown in Figure 8.

The space usage of $I(\hat{T})$ is closely related to the repetitiveness measures of $\hat{T}$, and a similar trend can be observed between Figure 6 and Figure 7. Under configurations with a high masked ratio, $\tau\lambda(I(\hat{T}))$ generally requires less space than $I(T)$, as observed in most cases for *dblp*, *english*, and *dna_real*, due to their smaller repetitiveness measures. In contrast, for *influenza* and *dna_pseudoreal*, the repetitiveness measures remain large across all configurations, resulting in a larger $I(\hat{T})$ and consequently a larger $\tau\lambda(I(\hat{T}))$. In such cases, we recommend replacing $I(\hat{T})$ with $I(T)$, that is, using $\tau\lambda(I(T))$ instead of $\tau\lambda(I(\hat{T}))$, to reduce the overall index size. Although this substitution still incurs additional space overhead from the XBWT, the XBWT can significantly improve query performance, especially when the query pattern contains no element of $MF_{\tau_l\tau_u\lambda}(T)$ and the search is performed using self-indexes that do not support efficient count queries, such as LZ77 and LPG. The detailed space breakdown of each component is reported in Table A.2.

To summarize the space-usage behavior, the masking-based approach is particularly effective for corpora with high masked ratios (at least 97% in our experiments), where it reduces overall space usage. Moreover, under our configuration, the space usage of the XBWT is relatively small (typically within 20% of the size of $\tau\lambda(I(T))$, and up to about 48% in a few cases). Consequently, the overall space usage of $\tau\lambda(I(\hat{T}))$ is largely determined by the repetitiveness measures of $\hat{T}$. However, these effects can only be evaluated after both $I(T)$ and $I(\hat{T})$ have been constructed.

4.4. Query Time

The results of the query time consumption are presented in Figure 9 (query patterns with at least one element of $MF_{\tau_l\tau_u\lambda}(T)$) and Figure 10 (query patterns with no element of $MF_{\tau_l\tau_u\lambda}(T)$). The values are expressed as percentage differences (%) relative to the query time of $I(T)$. To highlight the trend, each combination of τ_u and λ is colored according to its value: deeper orange indicates higher query time than the baseline, whereas deeper

blue indicates lower query time. Therefore, deeper blue corresponds to better performance in Figure 9 and Figure 10.

For query patterns that contain at least one element of $MF_{\tau_l\tau_u\lambda}(T)$, the query time of $\tau\lambda(I(\hat{T}))$ is generally expected to be higher than that of $I(T)$. The additional cost arises from the extra step of searching for a minimal factor using the XBWT (Step 2 in Algorithm 3). However, Figure 9 reveals several exceptions that exhibit the opposite trend. This behavior is caused by changes in the repetitiveness measures of $\hat{T}$, which in turn affect the search performed by the $I(\hat{T})$ (Step 3 in Algorithm 3), as discussed in the preliminaries.

In particular, the query time of the r-index increases as the repetitiveness measure r decreases. This explains why, in datasets where $\hat{T}$ has small r values, such as *dblp* and *dna_real*, the query time of $\tau\lambda(\text{r-index}(\hat{T}))$ increases more noticeably than in the other corpora. In contrast, the query time of LPG grows proportionally with the logarithm of its repetitiveness measure g. As a result, most instances of $\tau\lambda(\text{LPG}(\hat{T}))$ exhibit query-time trends that closely follow their space usage. The only exception occurs in the *dna_real* corpus, where $\hat{T}$ has a lower g value than T, yet its query time is higher. We attribute this behavior to the fact that queries on this corpus do not always exhibit the theoretical worst-case behavior, even though the worst-case query time complexity of $\text{LPG}(\hat{T})$ is lower than that of $\text{LPG}(T)$. In the LPG implementation [2], the primary-occurrence search often terminates earlier for $\text{LPG}(T)$ than for $\text{LPG}(\hat{T})$, resulting in shorter query times on this dataset. A more detailed investigation of this behavior is left for future work. As for LZ77, since its time complexity is influenced by both the number of phrases z and the parse height h, and no explicit relationship between these two parameters is known, it is difficult to observe a clear trend attributable to the masking process.

For query patterns that do not contain any element of $MF_{\tau_l\tau_u\lambda}(T)$, $\tau\lambda(I(\hat{T}))$ performs only a single query on the XBWT and immediately returns the empty set. Similarly, the r-index can also terminate early after verifying whether $\tau_l \leq \mathit{Count}(T, P) \leq \tau_u$ via a count query. However, since the time complexity of the r-index count query depends on the length of T, whereas the XBWT-based query does not, $\tau\lambda(\text{r-index}(\hat{T}))$ requires less query time on larger corpora, such as *influenza* and *dna_pseudoreal*.

For LZ77 and LPG, whose count queries rely on locate operations, we modified the implementations to terminate as soon as the number of reported locations exceeds τ_u. Even with this optimization, verifying whether a pattern is rare using the XBWT is generally much faster than executing locate queries on LZ77 or LPG. As a result, almost all instances of $\tau\lambda(\text{LZ77}(\hat{T}))$ and $\tau\lambda(\text{LPG}(\hat{T}))$ exhibit significantly lower query times, with the only exception being $\tau\lambda(\text{LZ77}(\hat{T}))$ on *dblp*. We attribute this exception to the relatively small z value and the small sample count of *dblp*. The small z value causes each locate query to be faster, as discussed in the preliminaries. The small sample count further leads to a small τ_u, which allows fewer locate operations to be executed before early termination, resulting in a shorter query time for $\text{LZ77}(T)$. In contrast, *english*, which has the same sample count as *dblp* but a larger z value, exhibits a different trend in query time. Furthermore, *influenza* and *dna_pseudoreal*, whose sample counts are much larger, show a substantially larger difference in query time.

In summary, these results suggest that $\tau\lambda$(r-index($\hat{T}$)) generally requires more query time compared to r-index(T). However, for indexes that do not support efficient count queries, such as LZ77 and LPG, $\tau\lambda$(LZ77($\hat{T}$)) and $\tau\lambda$(LPG($\hat{T}$)) often result in significantly reduced query time than LZ77(T) and LPG(T), particularly when the sample count is large.

4.5. Space-Time Trade-offs among Indexes

Figure 11 compares the different indexes across corpora. We set $\tau_l = 1$, $\tau_u \in \{1\%, 4\%\}$ of the sample count, and $\lambda = 200$. Among all corpora, r-index(T) or $\tau\lambda$(r-index($\hat{T}$)) dominate in query time, no matter whether the query patterns contain any element of $MF_{\tau_l\tau_u\lambda}(T)$ or not. However, $\tau\lambda$(LZ77($\hat{T}$)) (resp. $\tau\lambda$(LPG($\hat{T}$))) outperforms LZ77(T) (resp. LPG(T)) in most cases, especially when the query patterns contain no element of $MF_{\tau_l\tau_u\lambda}(T)$. If the space is concerned, LZ77(T) or $\tau\lambda$(LZ77($\hat{T}$)) outperforms others among all corpora.

In summary, when both space usage and query time are taken into account, $\tau\lambda$(LZ77($\hat{T}$)) is the best overall choice for most corpora. Compared with r-index(T), $\tau\lambda$(LZ77($\hat{T}$)) requires less space while incurring only a modest increase in query time in all corpora. Compared with LPG(T), $\tau\lambda$(LZ77($\hat{T}$)) requires both less space and less query time in every corpus. Compared with LZ77(T), although $\tau\lambda$(LZ77($\hat{T}$)) might require more space in some corpora (*english*, *influenza*, and *dna_pseudoreal*), it can significantly reduce query time, especially when the query patterns contain no element of $MF_{\tau_l\tau_u\lambda}(T)$.

5. Conclusion

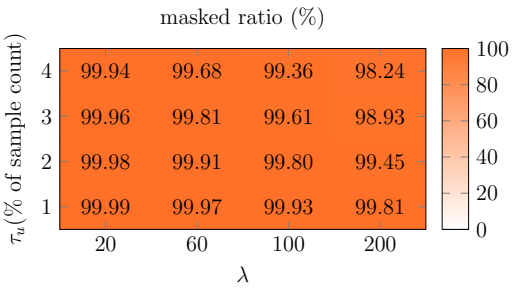
In this paper, we introduced the concept of rare information within a collection of similar strings, which holds promise for applications such as drug discovery and rare disease analysis. We also developed a near-linear time method to identify such rare information and proposed a framework to utilize it for future queries. This framework can be combined with any self-index and can reduce space usage on highly repetitive corpora.

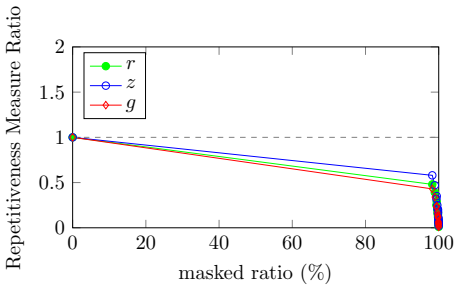
Experimental results demonstrate that our index achieves competitive space-time trade-offs on highly repetitive corpora across three types of self-indexes. In particular, for indexes that do not support efficient count queries, such as LZ77 and LPG, our method provides a significantly faster solution, often by orders of magnitude, while incurring only a modest space overhead in certain corpora. For another corpus, such as the real-world human DNA sequence, the $\tau\lambda$-index even requires less space than the three self-indexes compared in our experiments.

Several open problems remain for future work. First, the $\tau\lambda$-index built on a self-index may use more space than the self-index itself on certain corpora. At present, it is impossible to predict which index will be more space-efficient without constructing both $I(T)$ and $I(\hat{T})$. Developing techniques to estimate space usage in advance would therefore be valuable. Second, our framework requires the AC-automaton as auxiliary data structure, which introduces additional space and query-time overhead. It remains an open question whether this component can be eliminated or replaced with a lighter alternative.

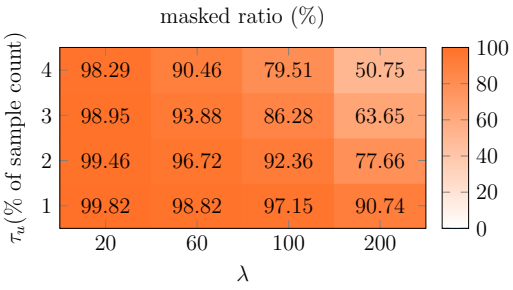
In summary, although our method does not consistently outperform all existing self-indexes, it offers strong potential, particularly on highly repetitive corpora in real-world applications such as human genomic sequences. This research also provides a new perspective on reducing space usage by focusing on the information that is truly necessary.

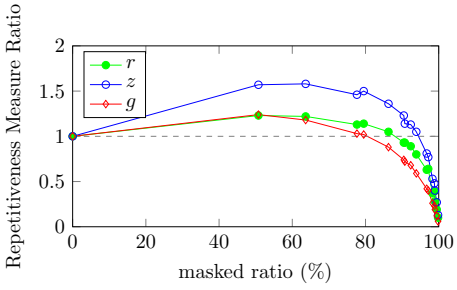
dblp



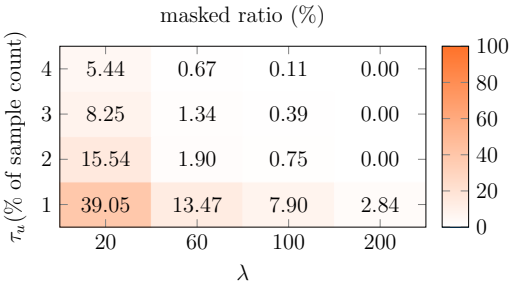


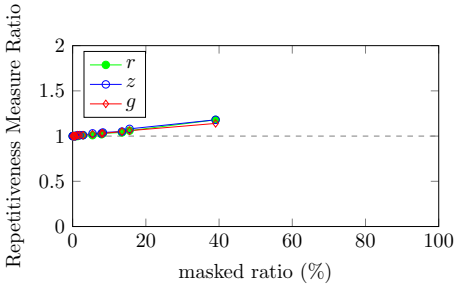
english



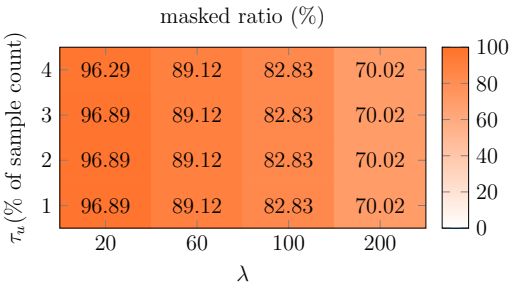


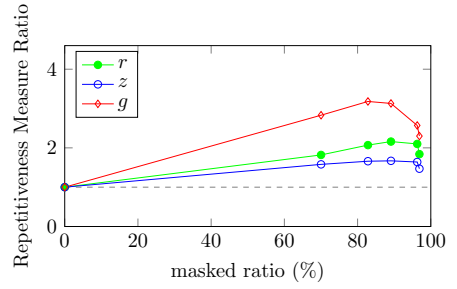
influenza





dna_pseudoreal





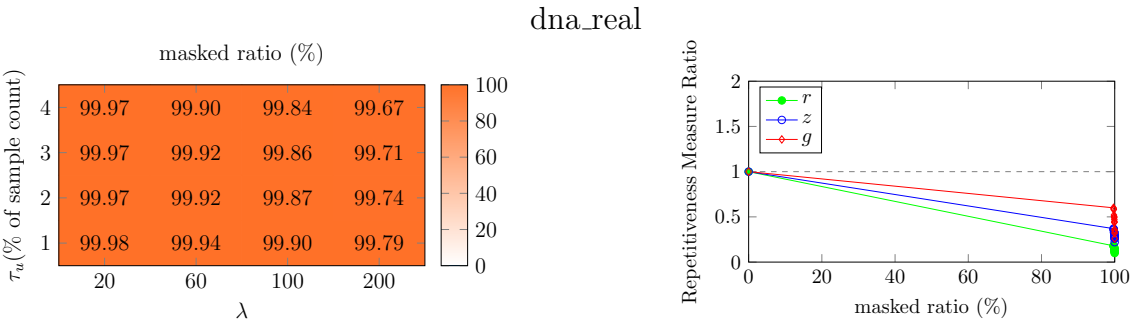


Figure 6: Masked ratios of the $\tau\lambda$-index on different corpora under various parameter settings, along with the normalized repetitiveness measures of the r-index, LZ77, and LPG.

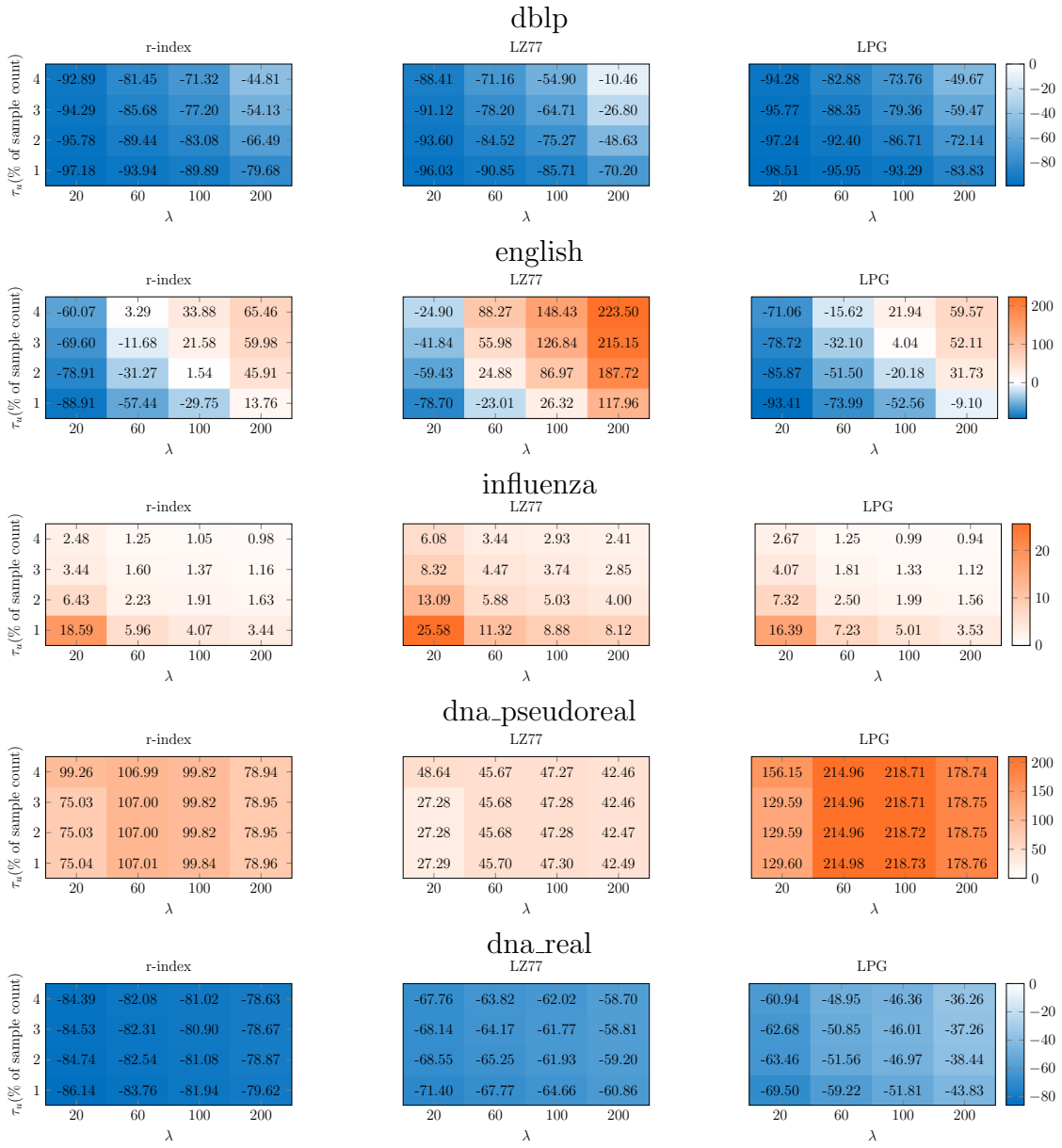
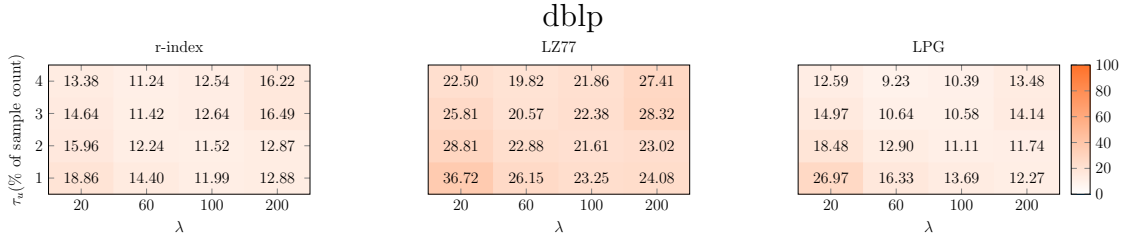


Figure 7: Serialized space (bytes/char) of the $\tau\lambda$-indexes with different underlying masked self-indexes, expressed as percentage differences (%) relative to the original indexes.



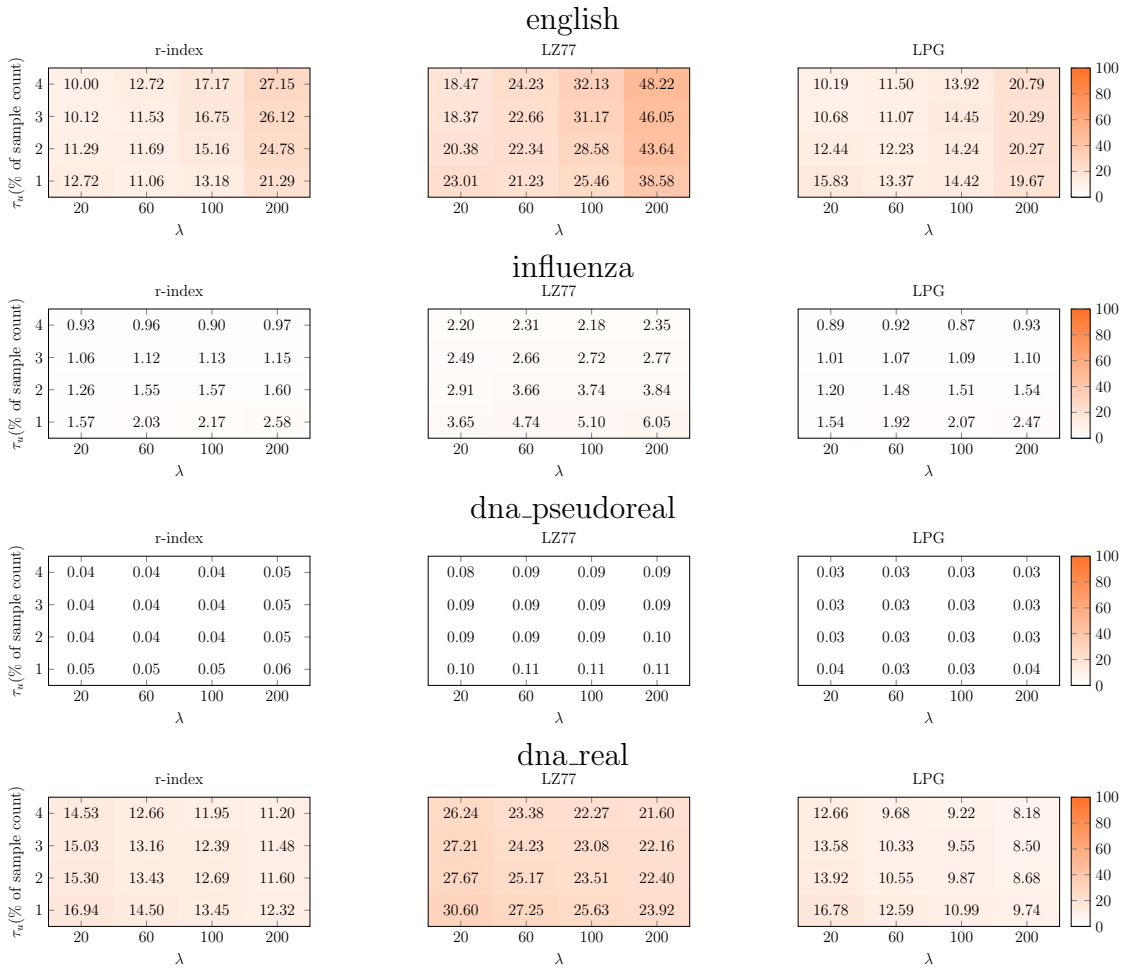
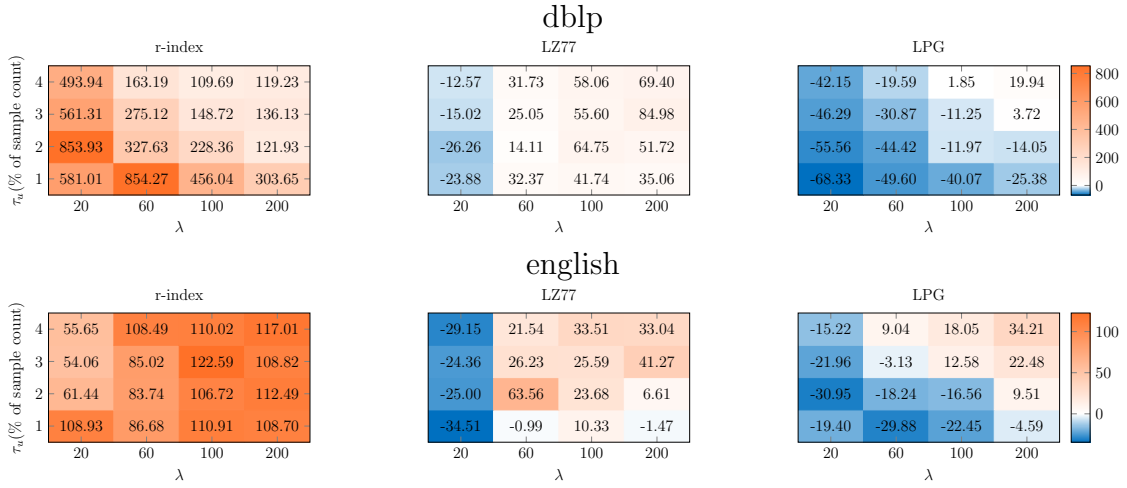


Figure 8: Space usage of XBWT as a percentage (%) of the $\tau\lambda$-index. The value is computed as space(XBWT)/space($\tau\lambda$-index).



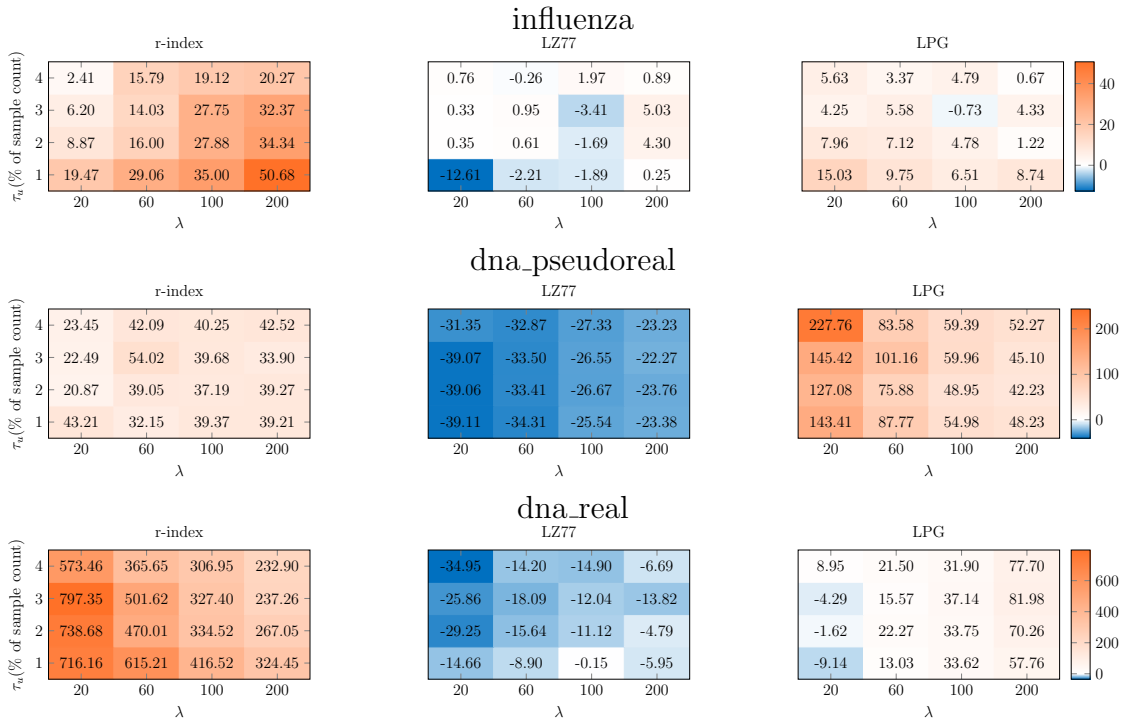
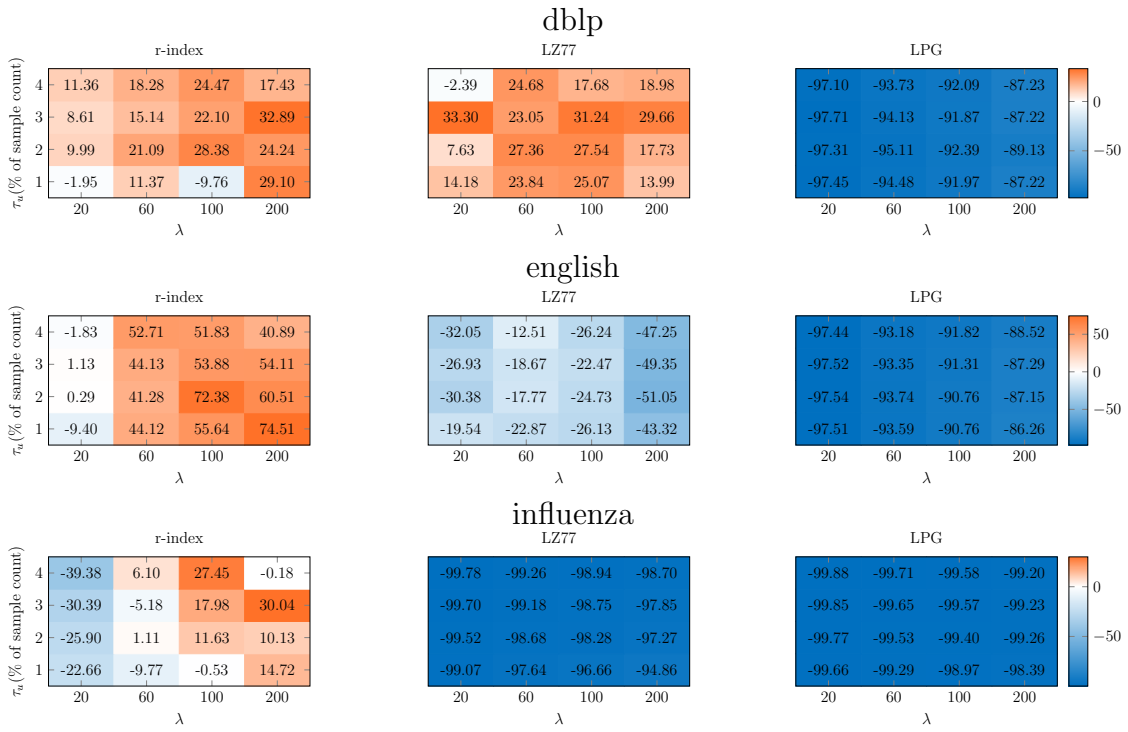


Figure 9: Location time consumption ($\mu s/Count(T,P)$) of the $\tau\lambda$-indexes with different underlying masked self-indexes, expressed as percentage differences (%) relative to the original indexes, based on query patterns with at least one element of $MF_{\tau_l\tau_u\lambda}(T)$.



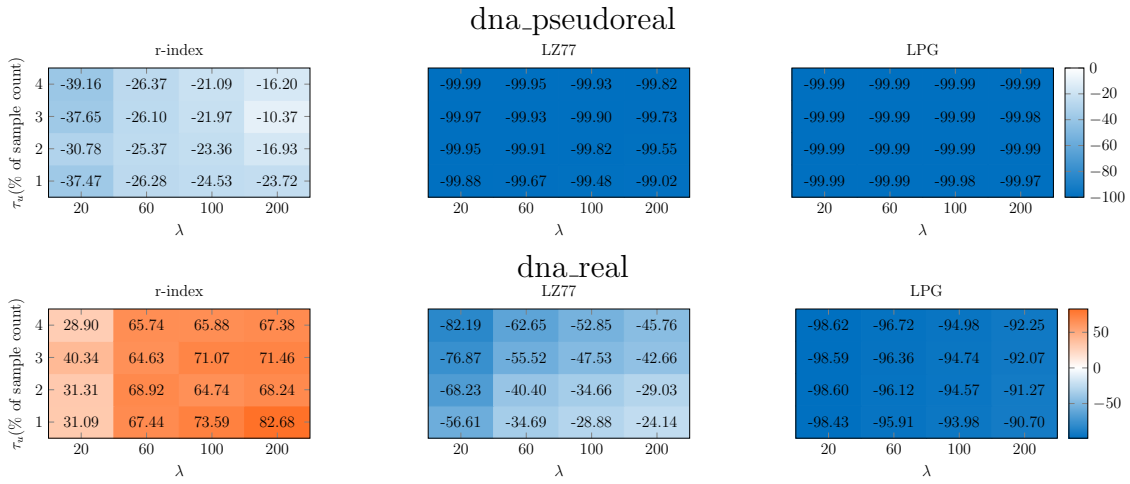


Figure 10: Location time consumption (μs/pattern) of the $\tau\lambda$-indexes with different underlying masked self-indexes, expressed as percentage differences (%) relative to the original indexes, based on query patterns with no element of $MF_{\tau_l\tau_u\lambda}(T)$. For such queries, since the $\tau\lambda$-index returns the empty set, the total time is divided by 100 to compute the time per query.

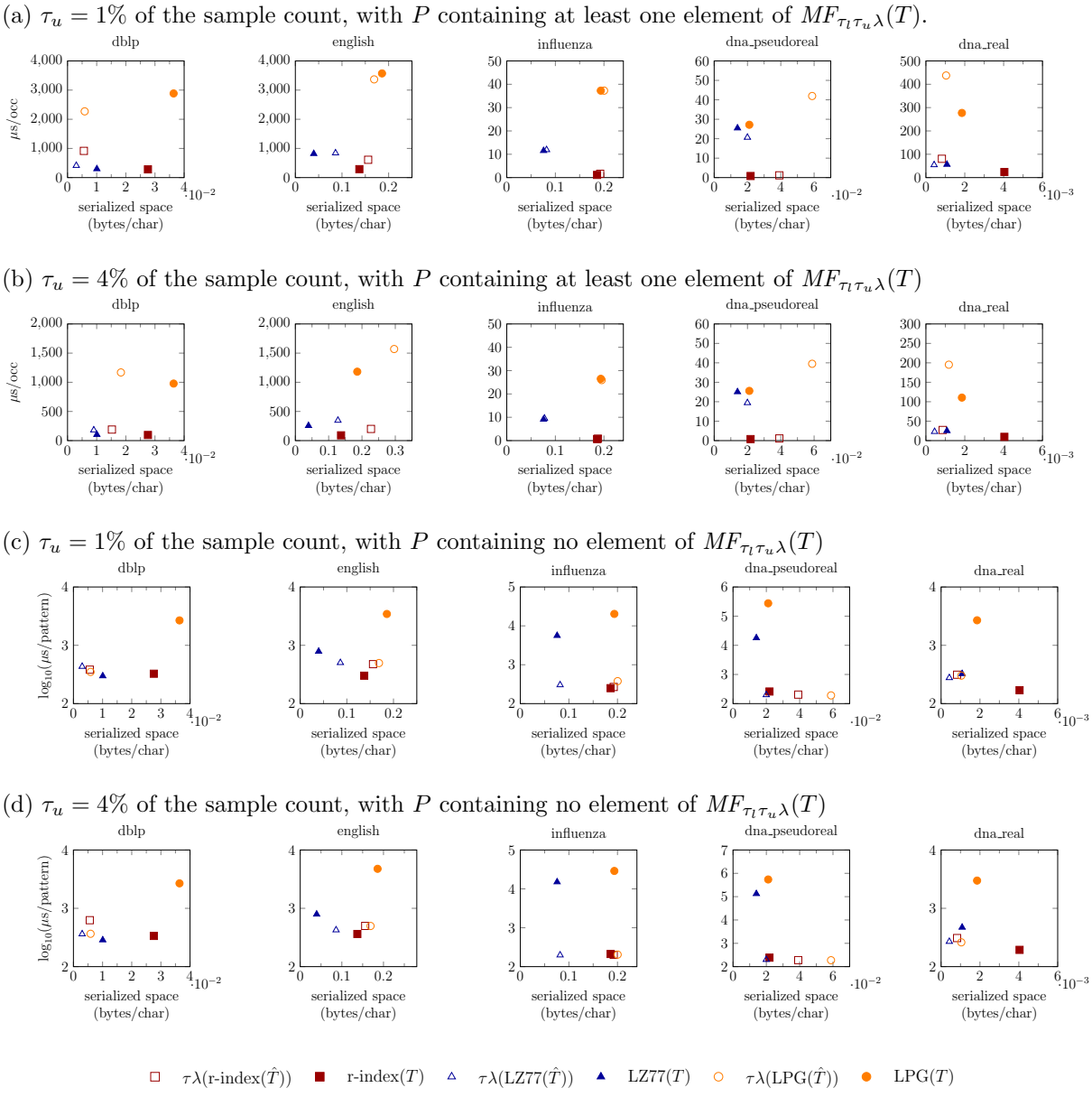


Figure 11: Comparison of query time and serialized space for $\tau_l = 1$ and $\lambda = 200$. We vary $\tau_u \in \{1\%, 4\%\}$ of the sample count and whether P contains elements of $MF_{\tau_l\tau_u\lambda}(T)$. Each subfigure (a)-(d) corresponds to a unique combination of these parameters. Query time in (a)-(b) is normalized by the sum of $Count(T, P)$, and in (c)-(d) divided by 100 to obtain the average time per query.

Acknowledgements. This research was supported by JSPS KAKENHI with grant numbers JP25K21150 and JP23H04378.

References

[1] F. Claude, A. Fariña, M. A. Martínez-Prieto, G. Navarro, Universal Indexes for Highly Repetitive Document Collections, Information Systems 61 (2016) 1–23. doi:10.1016/j.is.2016.04.002.

[2] D. Díaz-Domínguez, G. Navarro, A. Pacheco, An LMS-based Grammar Self-Index with Local Consistency Properties, in: String Processing and Information Retrieval, Vol. 12944, 2021, pp. 100–113. doi:10.1007/978-3-030-86692-1_9.

[3] T. Gagie, G. Navarro, N. Prezza, Optimal-Time Text Indexing in BWT-Runs Bounded Space, in: ACM-SIAM Symposium on Discrete Algorithms, 2018, pp. 1459—1477. doi:10.1137/1.9781611975031.96.

[4] I. H. G. Consortium, Initial Sequencing and Analysis of the Human Genome, Nature 409 (2001) 860–921. doi:10.1038/35057062.

[5] 1000 Genomes Project Consortium, A. Auton, L. D. Brooks, R. M. Durbin, E. P. Garrison, H. M. Kang, J. O. Korbel, J. L. Marchini, S. McCarthy, G. A. McVean, G. R. Abecasis, A global reference for human genetic variation, Nature 526 (7571) (2015) 68–74. doi:10.1038/nature15393.

[6] C. Turnbull, R. H. Scott, E. Thomas, L. Jones, N. Murugaesu, F. B. Pretty, D. Halai, E. Baple, C. Craig, A. Hamblin, et al., The 100 000 Genomes Project: Bringing Whole Genome Sequencing to the NHS, BMJ 361 (2018). doi:10.1136/bmj.k1687.

[7] D. L. Stern, The Genetic Causes of Convergent Evolution, Nature Reviews Genetics 14 (11) (2013) 751–764. doi:10.1038/nrg3483.

[8] E. M. McCreight, A Space-Economical Suffix Tree Construction Algorithm, Journal of the ACM 23 (2) (1976) 262–272. doi:10.1145/321941.321946.

[9] P. Weiner, Linear Pattern Matching Algorithms, in: Annual Symposium on Switching Automata Theory, 1973, pp. 1–11. doi:10.1109/SWAT.1973.13.

[10] P. Ferragina, G. Manzini, Indexing Compressed Text, Journal of the ACM 52 (4) (2005) 552–581. doi:10.1145/1082036.1082039.

[11] M. Burrows, D. J. Wheeler, A Block Sorting Lossless Data Compression Algorithm, Tech. rep., Digital Equipment Corporation (1994).

[12] P. Ferragina, G. Manzini, V. Mäkinen, G. Navarro, Compressed representations of sequences and full-text indexes, ACM Trans. Algorithms 3 (2007) 20–es. doi:10.1145/1240233.1240243.

[13] L. Kruglyak, D. A. Nickerson, Variation is the Spice of Life, Nature Genetics 27 (3) (2001) 234–236. doi:10.1038/85776.

[14] J. C. Venter, M. D. Adams, E. W. Myers, P. W. Li, R. J. Mural, et al., The Sequence of the Human Genome, Science 291 (5507) (2001) 1304–1351. doi:10.1126/science.1058040.

[15] G. Navarro, Indexing Highly Repetitive String Collections, Part I: Repetitiveness Measures, ACM Comput. Surv. 54 (2) (2021) 1–31. doi:10.1145/3434399.

[16] S. Kreft, G. Navarro, Self-indexing Based on LZ77, in: Combinatorial Pattern Matching, 2011, pp. 41–54. doi:10.1007/978-3-642-21458-5_6.

[17] H. Ferrada, D. Kempa, S. J. Puglisi, Hybrid Indexing Revisited, in: Proceedings of the Meeting on Algorithm Engineering and Experiments (ALENEX), 2018, pp. 1–8. doi:10.1137/1.9781611975055.1.

[18] D. Saad Nogueira Nunes, F. Louza, S. Gog, M. Ayala-Rincón, G. Navarro, A Grammar Compression Algorithm Based on Induced Suffix Sorting, in: 2018 Data Compression Conference, 2018, pp. 42–51. doi:10.1109/DCC.2018.00012.

[19] F. Claude, G. Navarro, A. Pacheco, Grammar-compressed Indexes with Logarithmic Search Time, Journal of Computer and System Sciences 118 (2021) 53–74. doi:10.1016/j.jcss.2020.12.001.

[20] D. Cobas, T. Gagie, G. Navarro, A Fast and Small Subsampled R-Index, in: Symposium on Combinatorial Pattern Matching, 2021, pp. 13:1–13:16. doi:10.4230/LIPIcs.CPM.2021.13.

[21] D. Kempa, T. Kociumaka, Collapsing the Hierarchy of Compressed Data Structures: Suffix Arrays in Optimal Compressed Space, in: IEEE Symposium on Foundations of Computer Science, 2023, pp. 1877–1886. doi:10.1109/FOCS57990.2023.00114.

[22] T. A. Manolio, F. S. Collins, N. J. Cox, D. B. Goldstein, L. A. Hindorff, D. J. Hunter, M. I. McCarthy, E. M. Ramos, L. R. Cardon, A. Chakravarti, et al., Finding the Missing Heritability of Complex Diseases, Nature 461 (7265) (2009) 747–753. doi:10.1038/nature08494.

[23] Y. Tateno, K. Ikeo, T. Imanishi, H. Watanabe, T. Endo, Y. Yamaguchi, Y. Suzuki, K. Takahashi, K. Tsunoyama, M. Kawai, Y. Kawanishi, K. Naitou, T. Gojobori, Evolutionary Motif and Its Biological and Structural Significance, Journal of Molecular Evolution 44 Suppl 1 (1997) S38–S43. doi:10.1007/pl00000056.

[24] S. Altschul, W. Gish, W. Miller, E. Myers, D. J. Lipman, Basic Local Alignment Search Tool, Journal of Molecular Biology 215 (3) (1990) 403–410. doi:10.1016/S0022-2836(05)80360-2.

[25] L.-Q. Chen, C.-W. Tsao, J.-J. Deng, W.-K. Hon, K. Sadakane, $\tau\lambda$-Index: Locating Rare Patterns in Similar Strings, in: IEEE Data Compression Conference, 2024, pp. 233–242. doi:10.1109/DCC58796.2024.00031.

[26] A. V. Aho, M. J. Corasick, Efficient String Matching: An Aid to Bibliographic Search, Communications of the ACM 18 (6) (1975) 333–340. doi:10.1145/360825.360855.

[27] P. Ferragina, F. Luccio, G. Manzini, S. Muthukrishnan, Compressing and Indexing Labeled Trees, with Applications, Journal of the ACM 57 (1) (2009) 1–33. doi:10.1145/1613676.1613680.

[28] R. Grossi, A. Gupta, J. S. Vitter, High-order entropy-compressed text indexes, in: Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms, 2003, p. 841–850. doi:10.5555/644108.644250.

[29] G. Manzini, XBWT Tricks, in: Symposium on String Processing and Information Retrieval, Vol. 9954, 2016, pp. 80–92. doi:10.1007/978-3-319-46049-9_8.

[30] G. Navarro, K. Sadakane, Fully Functional Static and Dynamic Succinct Trees, ACM Trans. Algorithms 10 (3) (2014). doi:10.1145/2601073.

[31] G. Rosone, M. Sciortino, The Burrows-Wheeler Transform between Data Compression and Combinatorics on Words, in: The Nature of Computation. Logic, Algorithms, Applications, 2013, pp. 353–364. doi:10.1007/978-3-642-39053-1_42.

[32] V. Mäkinen, G. Navarro, Succinct Suffix Arrays Based on Run-Length Encoding, in: Symposium on Combinatorial Pattern Matching, 2005, pp. 45–56. doi:10.1007/11496656_5.

[33] J. Ziv, A. Lempel, A Universal Algorithm for Sequential Data Compression, IEEE Transactions on information theory 23 (3) (1977) 337–343. doi:10.1109/TIT.1977.1055714.

[34] S. Kreft, G. Navarro, On Compressing and Indexing Repetitive Sequences, Theoretical Computer Science 483 (2013) 115–133. doi:10.1016/j.tcs.2012.02.006.

[35] M. Charikar, E. Lehman, D. Liu, R. Panigrahy, M. Prabhakaran, A. Sahai, A. Shelat, The Smallest Grammar Problem, IEEE Transactions on Information Theory 51 (7) (2005) 2554–2576. doi:10.1109/TIT.2005.850116.

[36] L. Ilie, W. F. Smyth, Minimum Unique Substrings and Maximum Repeats, Fundamenta Informaticae 110 (1-4) (2011) 183–195. doi:10.3233/FI-2011-536.

[37] J. C. Na, A. Apostolico, C. S. Iliopoulos, K. Park, Truncated suffix trees and their application to data compression, Theoretical Computer Science 304 (1) (2003) 87–101. doi:10.1016/S0304-3975(03)00053-7.

[38] D. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology, Cambridge University Press, 1997.

[39] S. Gog, T. Beller, A. Moffat, M. Petri, From Theory to Practice: Plug and Play with Succinct Data Structures, in: Proceedings of the 13th International Symposium on Experimental Algorithms - Volume 8504, 2014, p. 326–337. doi:10.1007/978-3-319-07959-2_28.

[40] R. Sachidanandam, D. Weissman, S. C. Schmidt, J. M. Kakol, L. D. Stein, G. Marth, S. Sherry, J. C. Mullikin, B. J. Mortimore, D. L. Willey, et al., A Map of Human Genome Sequence Variation Containing 1.42 Million Single Nucleotide Polymorphisms, Nature 409 (6822) (2001) 928–933. doi:10.1038/35057149.

[41] Z. Zhao, Y.-X. Fu, D. Hewett-Emmett, E. Boerwinkle, Investigating Single Nucleotide Polymorphism (SNP) Density in the Human Genome and Its Implications for Molecular Evolution, Gene 312 (2003) 207–213. doi:10.1016/S0378-1119(03)00670-X.

[42] W. Shao, X. Li, M. U. Goraya, S. Wang, J.-L. Chen, Evolution of Influenza A Virus by Mutation and Re-Assortment, International Journal of Molecular Sciences 18 (8) (2017). doi:10.3390/ijms18081650.

Appendix A. Extra Data

Table A.2: Serialized space usage (MiB) of the XBWT, $I(T)$, and $I(\hat{T})$ across different configurations and corpora. The parameters are set as $\tau_l = 1$, $\tau_u \in \{1\%, 2\%, 3\%, 4\%\}$ of the sample count, and $\lambda \in \{20, 60, 100, 200\}$.

	dblp	*english*	*influenza*	*dna_pseudoreal*	*dna_real*
			(XBWT)		
$\tau_u = 1\%, \lambda = 20$	0.015	0.194	0.512	0.011	0.009
$\tau_u = 1\%, \lambda = 60$	0.024	0.646	0.589	0.013	0.010
$\tau_u = 1\%, \lambda = 100$	0.033	1.271	0.621	0.013	0.010
$\tau_u = 1\%, \lambda = 200$	0.072	3.323	0.731	0.013	0.010
$\tau_u = 2\%, \lambda = 20$	0.019	0.327	0.368	0.010	0.009
$\tau_u = 2\%, \lambda = 60$	0.036	1.103	0.434	0.011	0.009
$\tau_u = 2\%, \lambda = 100$	0.054	2.112	0.439	0.011	0.010
$\tau_u = 2\%, \lambda = 200$	0.119	4.963	0.447	0.011	0.010
$\tau_u = 3\%, \lambda = 20$	0.023	0.422	0.301	0.010	0.009
$\tau_u = 3\%, \lambda = 60$	0.045	1.397	0.311	0.011	0.009
$\tau_u = 3\%, \lambda = 100$	0.079	2.794	0.316	0.011	0.010
$\tau_u = 3\%, \lambda = 200$	0.209	5.735	0.319	0.011	0.010
$\tau_u = 4\%, \lambda = 20$	0.026	0.548	0.261	0.010	0.009
$\tau_u = 4\%, \lambda = 60$	0.057	1.803	0.267	0.011	0.009
$\tau_u = 4\%, \lambda = 100$	0.099	3.154	0.251	0.011	0.009
$\tau_u = 4\%, \lambda = 200$	0.247	6.165	0.269	0.011	0.010
r-index(T)	2.758	13.723	27.442	13.123	0.404
			(r-index($\hat{T}$))		
$\tau_u = 1\%, \lambda = 20$	0.063	1.329	32.031	22.959	0.046
$\tau_u = 1\%, \lambda = 60$	0.143	5.195	28.489	27.153	0.056
$\tau_u = 1\%, \lambda = 100$	0.245	8.370	27.938	26.211	0.063
$\tau_u = 1\%, \lambda = 200$	0.488	12.288	27.656	23.472	0.072
$\tau_u = 2\%, \lambda = 20$	0.098	2.568	28.840	22.959	0.052
$\tau_u = 2\%, \lambda = 60$	0.256	8.330	27.620	27.153	0.061
$\tau_u = 2\%, \lambda = 100$	0.413	11.822	27.528	26.211	0.067
$\tau_u = 2\%, \lambda = 200$	0.805	15.061	27.442	23.472	0.075
$\tau_u = 3\%, \lambda = 20$	0.134	3.750	28.084	22.959	0.053
$\tau_u = 3\%, \lambda = 60$	0.350	10.723	27.571	27.153	0.062
$\tau_u = 3\%, \lambda = 100$	0.549	13.890	27.502	26.211	0.068
$\tau_u = 3\%, \lambda = 200$	1.056	16.220	27.442	23.472	0.076
$\tau_u = 4\%, \lambda = 20$	0.170	4.932	27.860	26.139	0.054

(continues on next page)

	dblp	*english*	*influenza*	*dna_pseudoreal*	*dna_real*
$\tau_u = 4\%, \lambda = 60$	0.454	12.372	27.518	27.153	0.063
$\tau_u = 4\%, \lambda = 100$	0.692	15.218	27.480	26.211	0.067
$\tau_u = 4\%, \lambda = 200$	1.275	16.542	27.442	23.472	0.077
LZ77(T)	1.006	3.952	11.182	8.404	0.108
			(LZ77($\hat{T}$))		
$\tau_u = 1\%, \lambda = 20$	0.025	0.648	13.531	10.686	0.021
$\tau_u = 1\%, \lambda = 60$	0.068	2.397	11.859	12.231	0.025
$\tau_u = 1\%, \lambda = 100$	0.110	3.722	11.555	12.365	0.028
$\tau_u = 1\%, \lambda = 200$	0.228	5.291	11.359	11.961	0.032
$\tau_u = 2\%, \lambda = 20$	0.046	1.277	12.278	10.686	0.025
$\tau_u = 2\%, \lambda = 60$	0.120	3.833	11.405	12.231	0.028
$\tau_u = 2\%, \lambda = 100$	0.195	5.277	11.305	12.365	0.032
$\tau_u = 2\%, \lambda = 200$	0.398	6.408	11.182	11.961	0.034
$\tau_u = 3\%, \lambda = 20$	0.066	1.876	11.812	10.686	0.025
$\tau_u = 3\%, \lambda = 60$	0.174	4.768	11.370	12.231	0.029
$\tau_u = 3\%, \lambda = 100$	0.276	6.171	11.285	12.365	0.032
$\tau_u = 3\%, \lambda = 200$	0.528	6.720	11.182	11.961	0.035
$\tau_u = 4\%, \lambda = 20$	0.090	2.420	11.601	12.481	0.026
$\tau_u = 4\%, \lambda = 60$	0.233	5.638	11.300	12.231	0.030
$\tau_u = 4\%, \lambda = 100$	0.355	6.664	11.259	12.365	0.032
$\tau_u = 4\%, \lambda = 200$	0.654	6.621	11.182	11.961	0.035
LPG(T)	3.639	18.585	28.568	12.663	0.185
			(LPG($\hat{T}$))		
$\tau_u = 1\%, \lambda = 20$	0.040	1.030	32.738	29.063	0.047
$\tau_u = 1\%, \lambda = 60$	0.123	4.188	30.045	39.871	0.066
$\tau_u = 1\%, \lambda = 100$	0.211	7.545	29.378	40.347	0.079
$\tau_u = 1\%, \lambda = 200$	0.516	13.571	28.845	35.285	0.094
$\tau_u = 2\%, \lambda = 20$	0.082	2.299	30.290	29.063	0.058
$\tau_u = 2\%, \lambda = 60$	0.241	7.911	28.848	39.871	0.080
$\tau_u = 2\%, \lambda = 100$	0.430	12.722	28.699	40.347	0.088
$\tau_u = 2\%, \lambda = 200$	0.895	19.519	28.568	35.285	0.104
$\tau_u = 3\%, \lambda = 20$	0.131	3.532	29.429	29.063	0.060
$\tau_u = 3\%, \lambda = 60$	0.379	11.222	28.773	39.871	0.082
$\tau_u = 3\%, \lambda = 100$	0.671	16.542	28.631	40.347	0.090
$\tau_u = 3\%, \lambda = 200$	1.266	22.533	28.568	35.285	0.106
$\tau_u = 4\%, \lambda = 20$	0.182	4.830	29.069	32.426	0.063
$\tau_u = 4\%, \lambda = 60$	0.565	13.879	28.658	39.871	0.085
$\tau_u = 4\%, \lambda = 100$	0.856	19.507	28.600	40.347	0.090

(continues on next page)

	dblp	*english*	*influenza*	*dna_pseudoreal*	*dna_real*
$\tau_u = 4\%, \lambda = 200$	1.585	23.492	28.568	35.285	0.108